Booting

Backward compatibility: x86 segments, descriptor tables
BIOS, Bootloader, Read/write disk sectors

Abhilash Jindal

Reference. xv6 book: Appendix B

Agenda

Agenda

- Boot a minimal OS!

Agenda

- Boot a minimal OS!
- Understand code

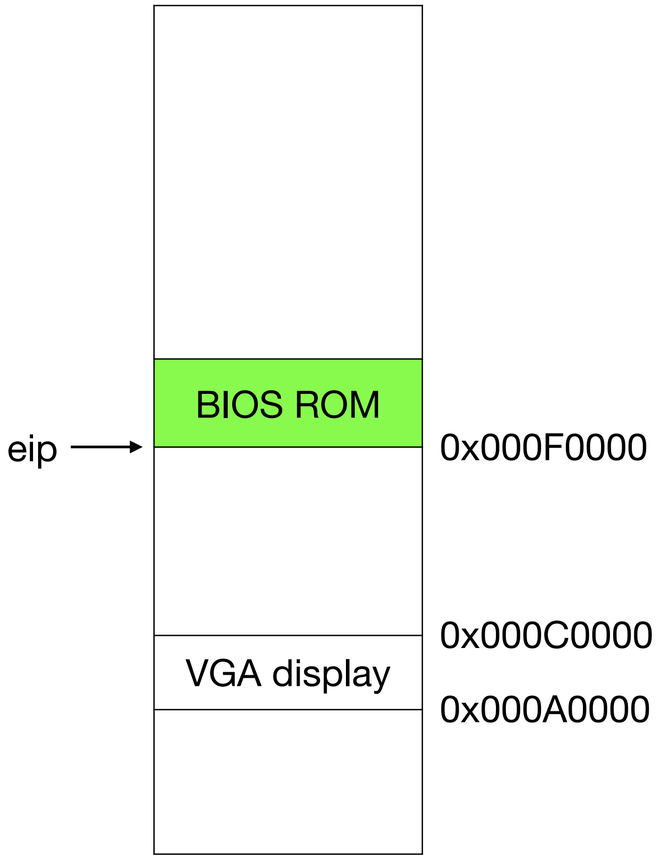
Agenda

- Boot a minimal OS!
- Understand code
  - Learn what happens when we power on a computer

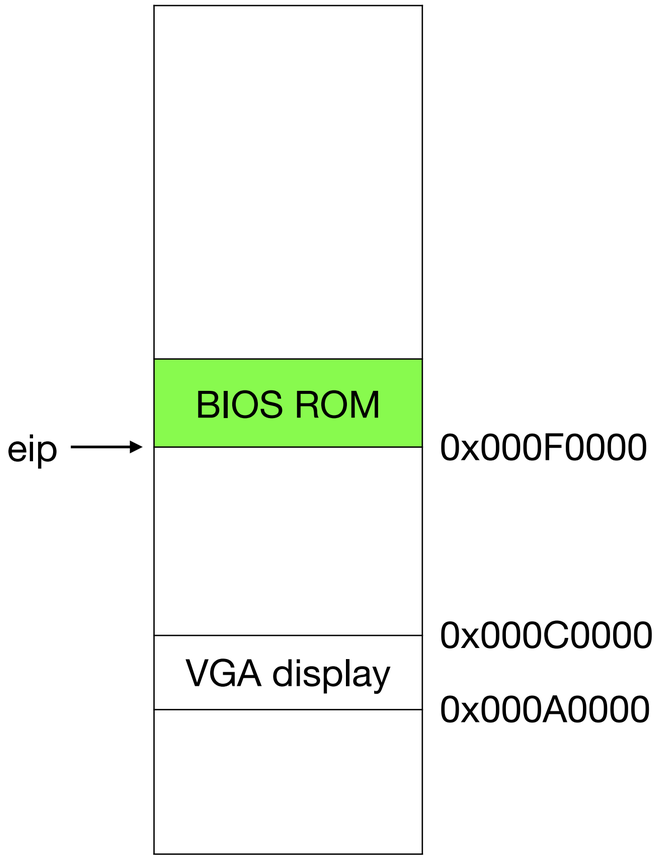
Agenda

- Boot a minimal OS!
- Understand code
  - Learn what happens when we power on a computer
  - Learn few more x86 details required for booting

Boot up sequence (1): BIOS

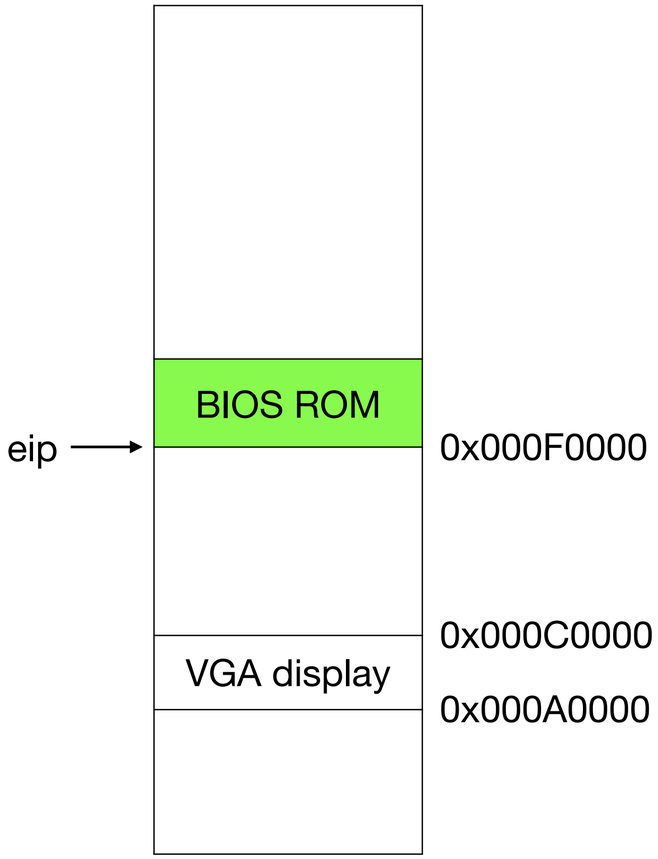


Boot up sequence (1): BIOS



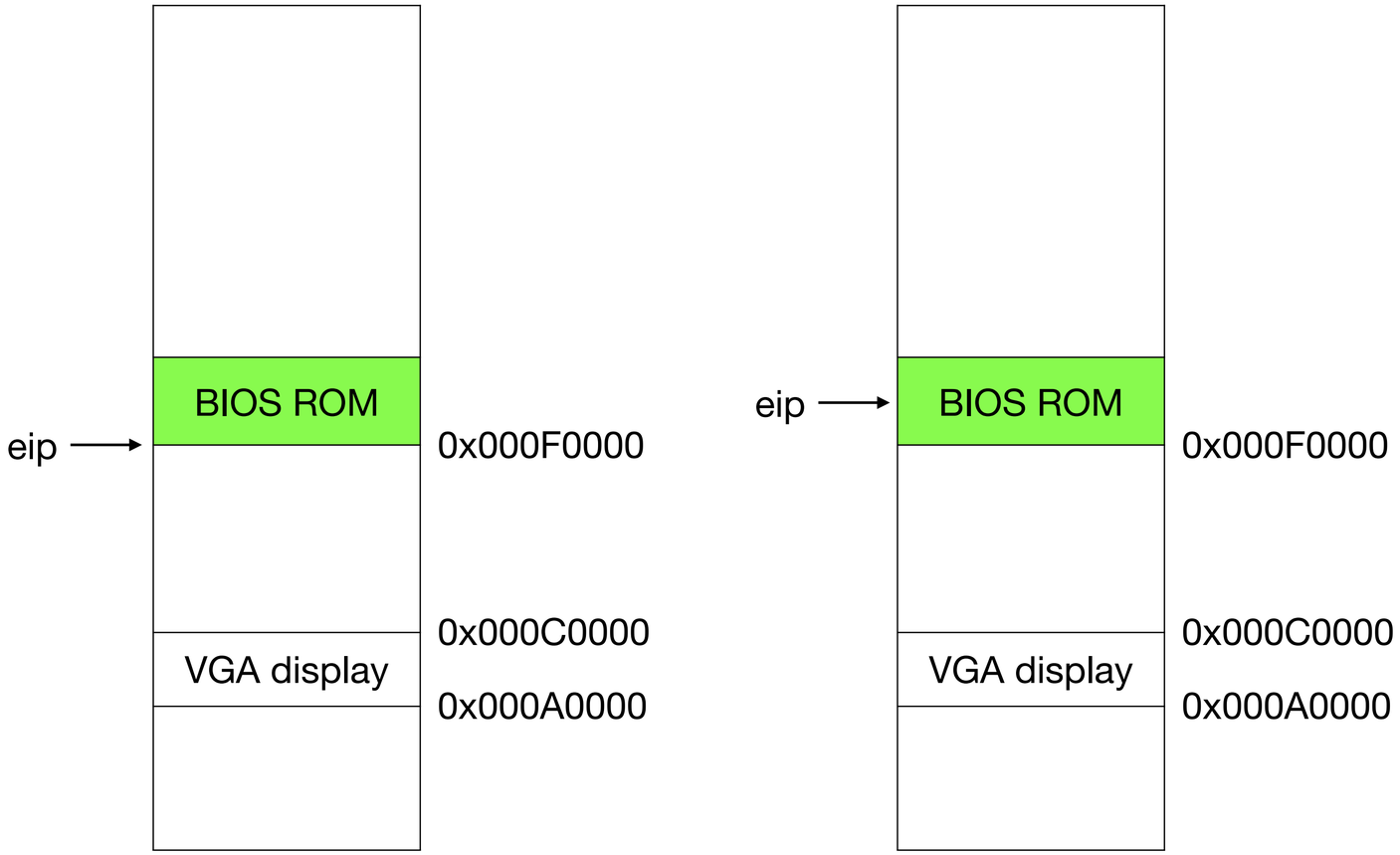
- For backward compatibility, PC boots in 16-bit mode

Boot up sequence (1): BIOS



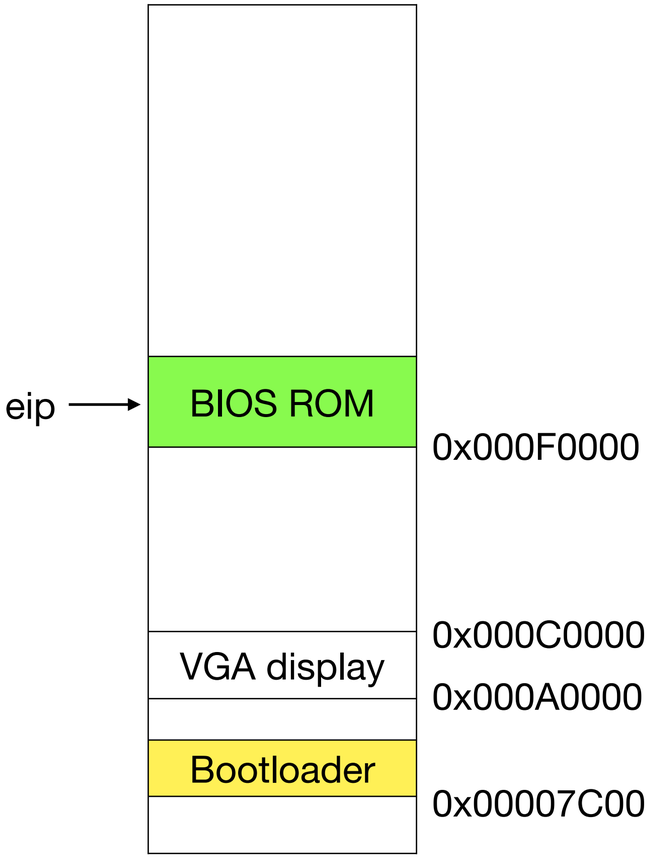
- For backward compatibility, PC boots in 16-bit mode
- BIOS does initial hardware check: are CPUs, memory, disk functional?

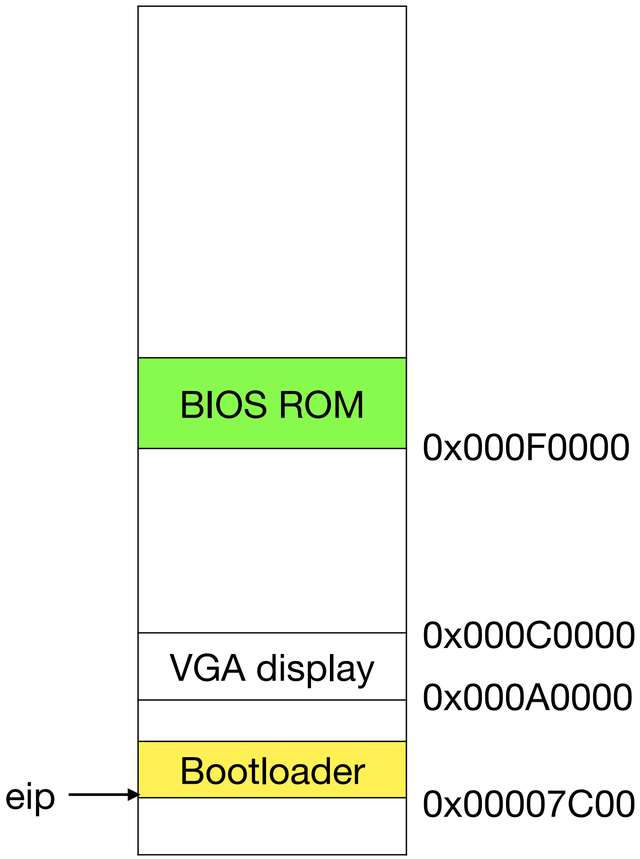
Boot up sequence (1): BIOS



- For backward compatibility, PC boots in 16-bit mode
- BIOS does initial hardware check: are CPUs, memory, disk functional?

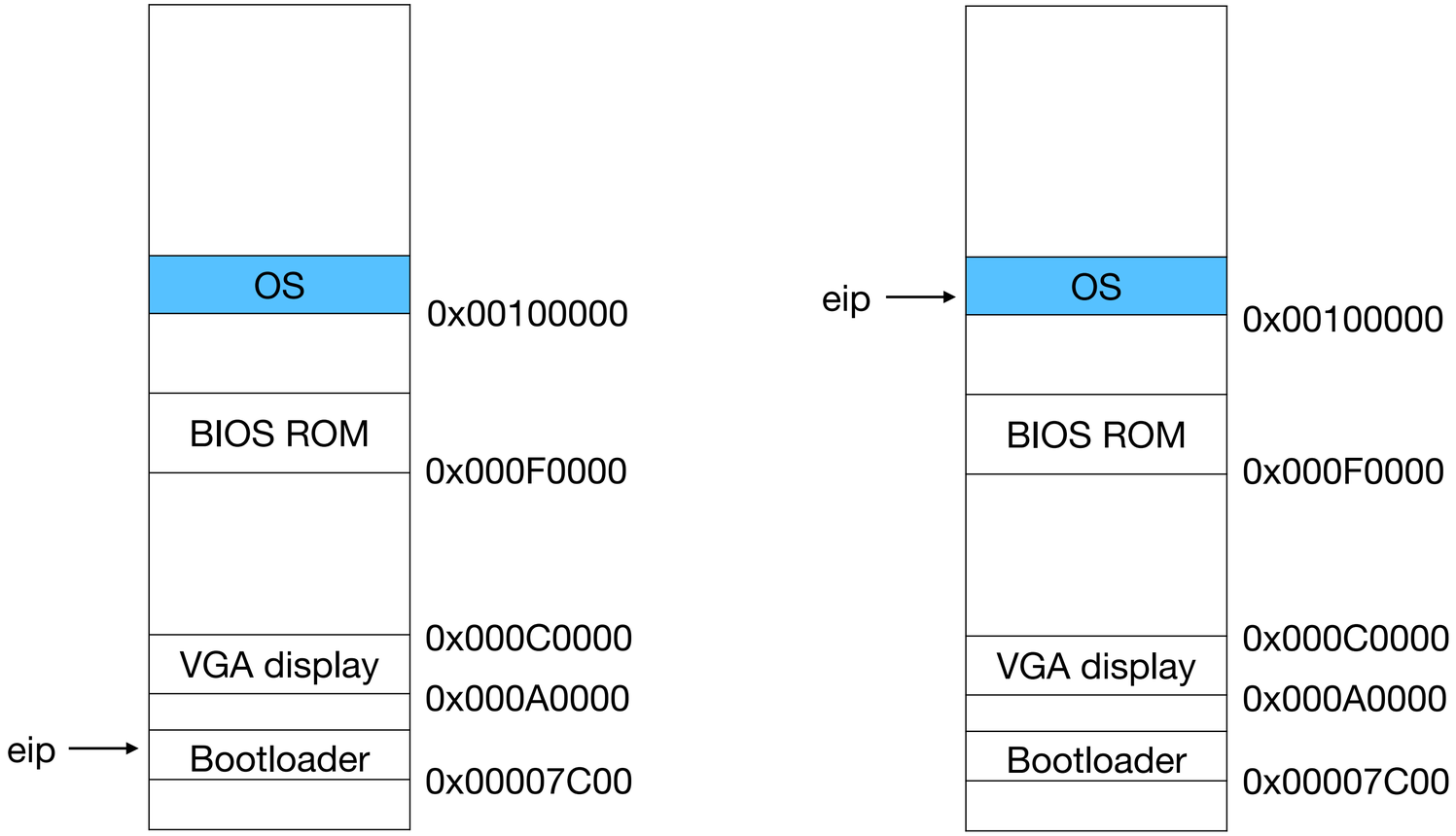
Boot up sequence (2): Bootloader



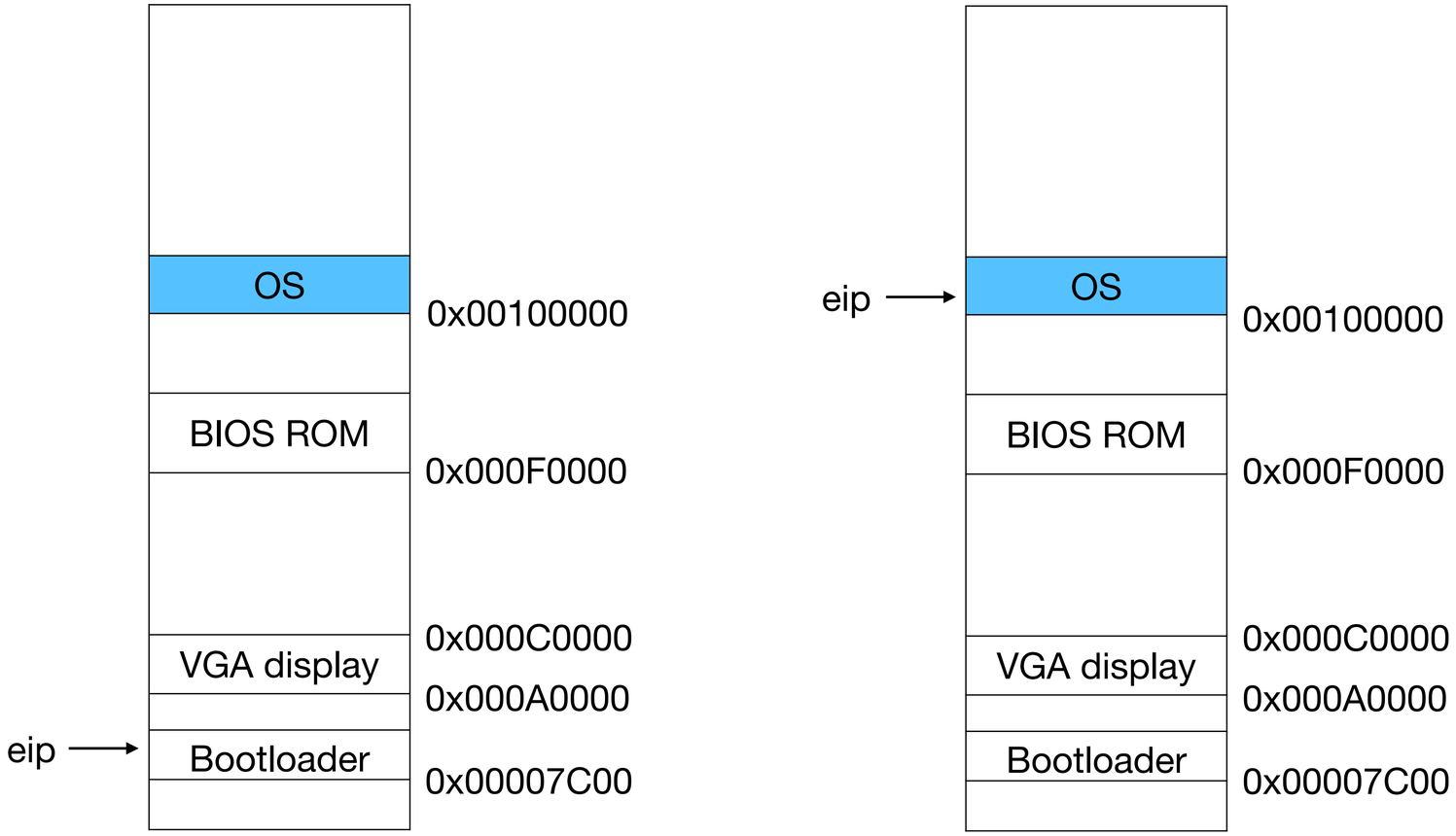


- BIOS loads first disk sector (512 bytes) at 0x7c00 and gives control
  - We need to write a boot loader that fits in the first sector of disk
  - We need to tell gcc that boot loader will load at 0x7c00 (virtual memory to physical memory translation is not setup yet!)
- Boot loader changes to 32 bit mode

Boot up sequence (3): OS

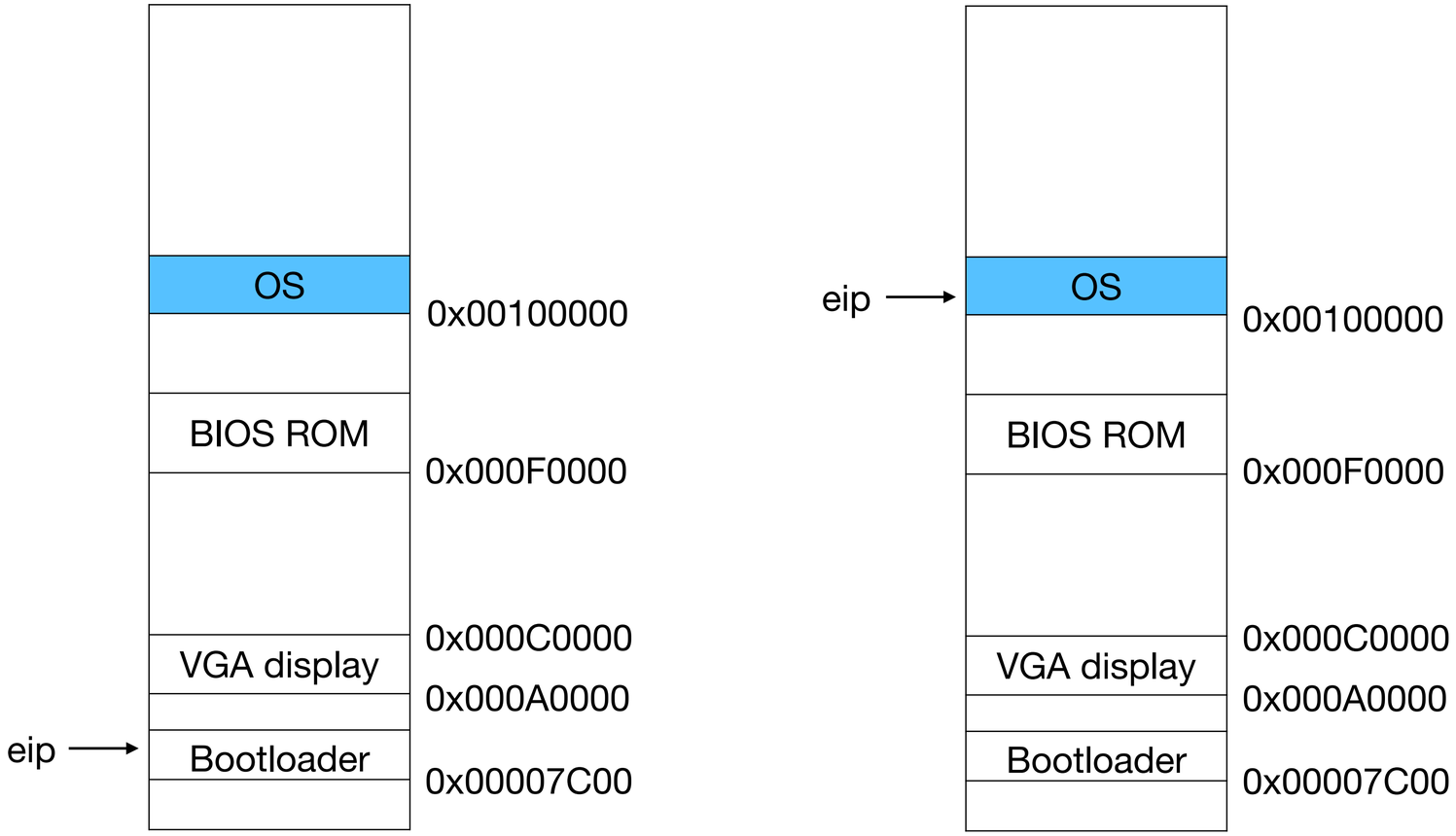


Boot up sequence (3): OS



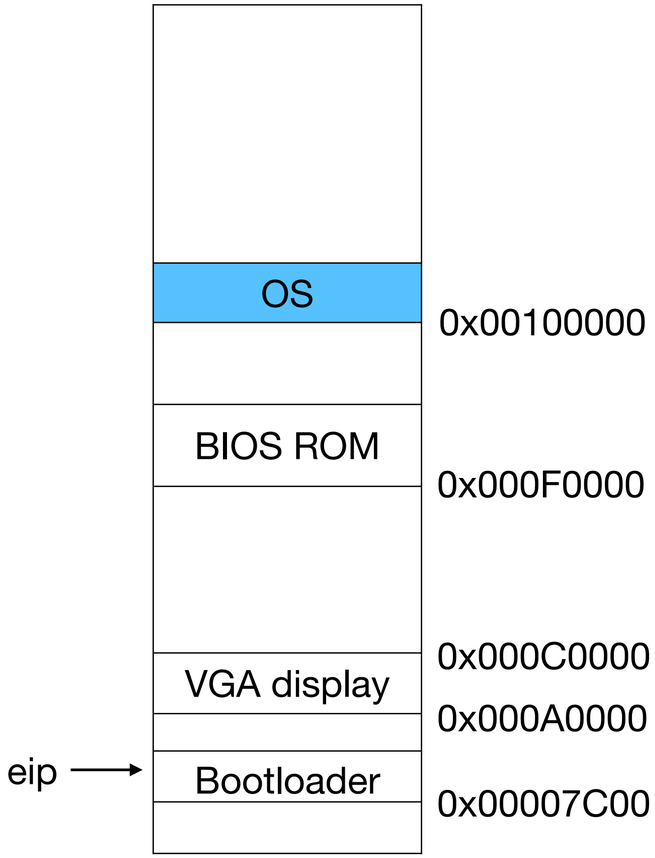
- gcc prepares OS image in Executable and Linkable Format (ELF)

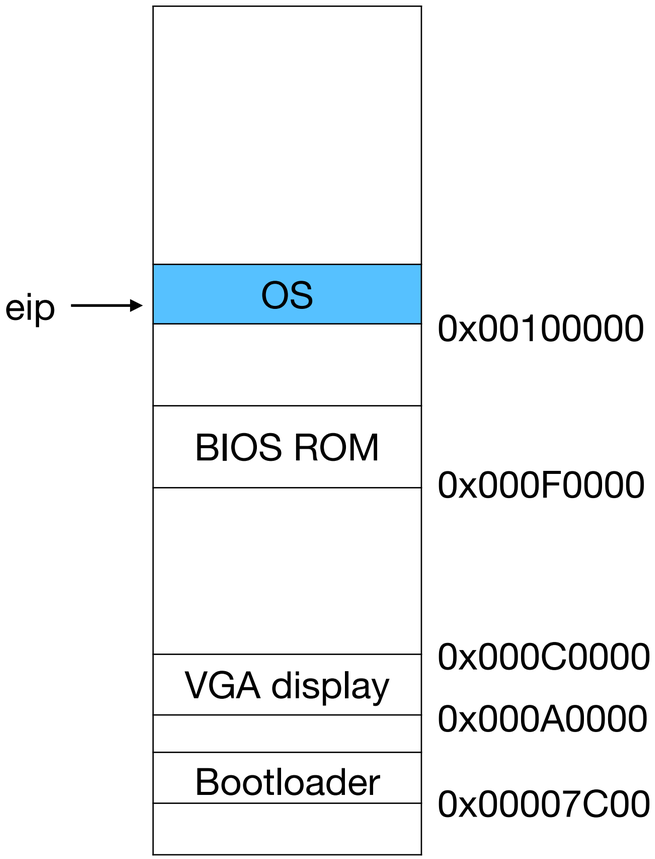
Boot up sequence (3): OS



- gcc prepares OS image in Executable and Linkable Format (ELF)
- Bootloader copies OS image starting from disk sector 1 to 0x00100000 and transfers control to it

Boot up sequence (3): OS





- gcc prepares OS image in Executable and Linkable Format (ELF)
- Bootloader copies OS image starting from disk sector 1 to 0x00100000 and transfers control to it
- We need to tell gcc that OS image will be loaded at 0x00100000

Backward compatibility

Backward compatibility

- When boot loader gets control, the CPU is in *16-bit mode*

Backward compatibility

- When boot loader gets control, the CPU is in *16-bit mode*
  - This is for backward compatibility. OS written for 16-bit mode should just work for 32-bit and 64-bit machines

Backward compatibility

- When boot loader gets control, the CPU is in *16-bit mode*
  - This is for backward compatibility. OS written for 16-bit mode should just work for 32-bit and 64-bit machines
- Bootloader explicitly switches from 16-bit mode to 32-bit mode

Backward compatibility

- When boot loader gets control, the CPU is in *16-bit mode*
  - This is for backward compatibility. OS written for 16-bit mode should just work for 32-bit and 64-bit machines
- Bootloader explicitly switches from 16-bit mode to 32-bit mode
- Understand how hardware provides backward compatibility and some historical details of 16-bit architecture

16-bit registers

General-Purpose Registers

31 … 16	15 … 8	7 … 0	16-bit	32-bit
	AH	AL	AX	EAX
	BH	BL	BX	EBX
	CH	CL	CX	ECX
	DH	DL	DX	EDX
	BP			EBP
	SI			ESI
	DI			EDI
	SP			ESP

Figure 3-5. Alternate General-Purpose Register Names

16-bit registers

- All registers were 16-bit on 16-bit CPU

General-Purpose Registers

31 … 16	15 … 8	7 … 0	16-bit	32-bit
	AH	AL	AX	EAX
	BH	BL	BX	EBX
	CH	CL	CX	ECX
	DH	DL	DX	EDX
	BP			EBP
	SI			ESI
	DI			EDI
	SP			ESP

Figure 3-5. Alternate General-Purpose Register Names

16-bit registers

- All registers were 16-bit on 16-bit CPU
- movw %ax, %bx : move low 16 bits of %eax into 16 bits of %ebx

General-Purpose Registers

31 … 16	15 … 8	7 … 0	16-bit	32-bit
	AH	AL	AX	EAX
	BH	BL	BX	EBX
	CH	CL	CX	ECX
	DH	DL	DX	EDX
	BP			EBP
	SI			ESI
	DI			EDI
	SP			ESP

Figure 3-5. Alternate General-Purpose Register Names

16-bit registers

- All registers were 16-bit on 16-bit CPU
- movw %ax, %bx : move low 16 bits of %eax into 16 bits of %ebx
- movb %al, %bl : move low 8 bits of %eax into 8 bits of %ebx

General-Purpose Registers

31 … 16	15 … 8	7 … 0	16-bit	32-bit
	AH	AL	AX	EAX
	BH	BL	BX	EBX
	CH	CL	CX	ECX
	DH	DL	DX	EDX
	BP			EBP
	SI			ESI
	DI			EDI
	SP			ESP

Figure 3-5. Alternate General-Purpose Register Names

16-bit registers

- All registers were 16-bit on 16-bit CPU
- movw %ax, %bx : move low 16 bits of %eax into 16 bits of %ebx
- movb %al, %bl : move low 8 bits of %eax into 8 bits of %ebx
- When CPU is in 16-bit mode, 32-bit and 64-bit machines continue to support same opcode for these instructions

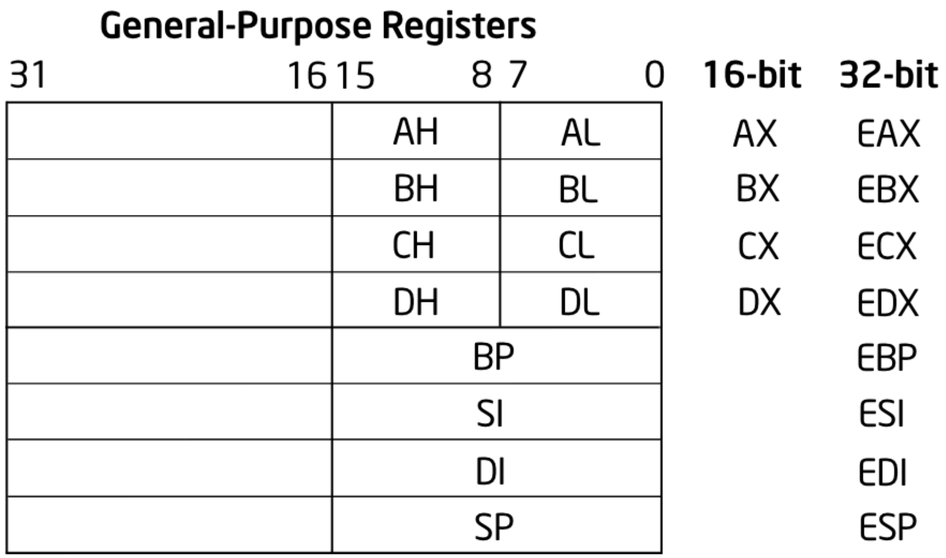


Figure 3-5. Alternate General-Purpose Register Names

Segment registers

Segment registers

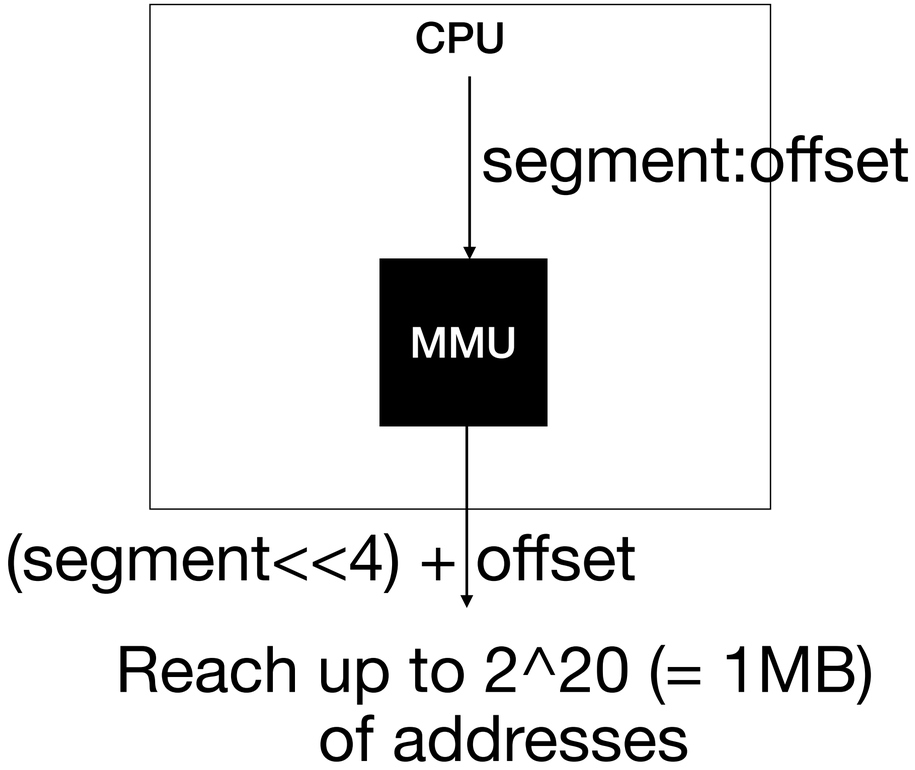
- 16-bit registers can only point up to 2^16 (=64KB) addresses in DRAM

Segment registers

- 16-bit registers can only point up to 2^16 (=64KB) addresses in DRAM
- Full address = (segment registers : offset)

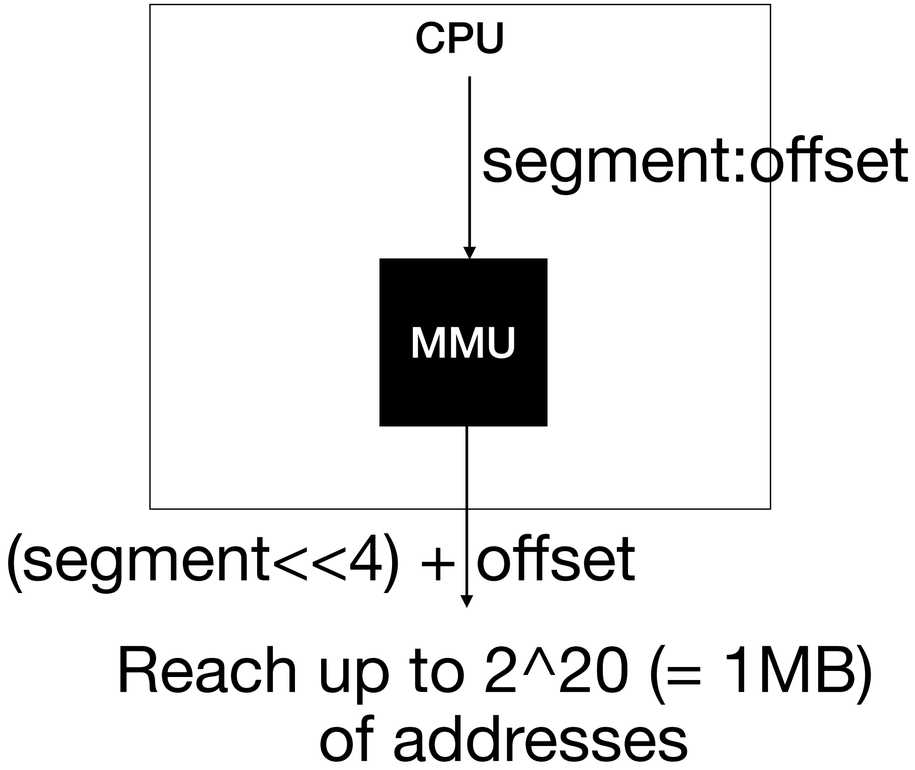
Segment registers

- 16-bit registers can only point up to 2^16 (=64KB) addresses in DRAM
- Full address = (segment registers : offset)



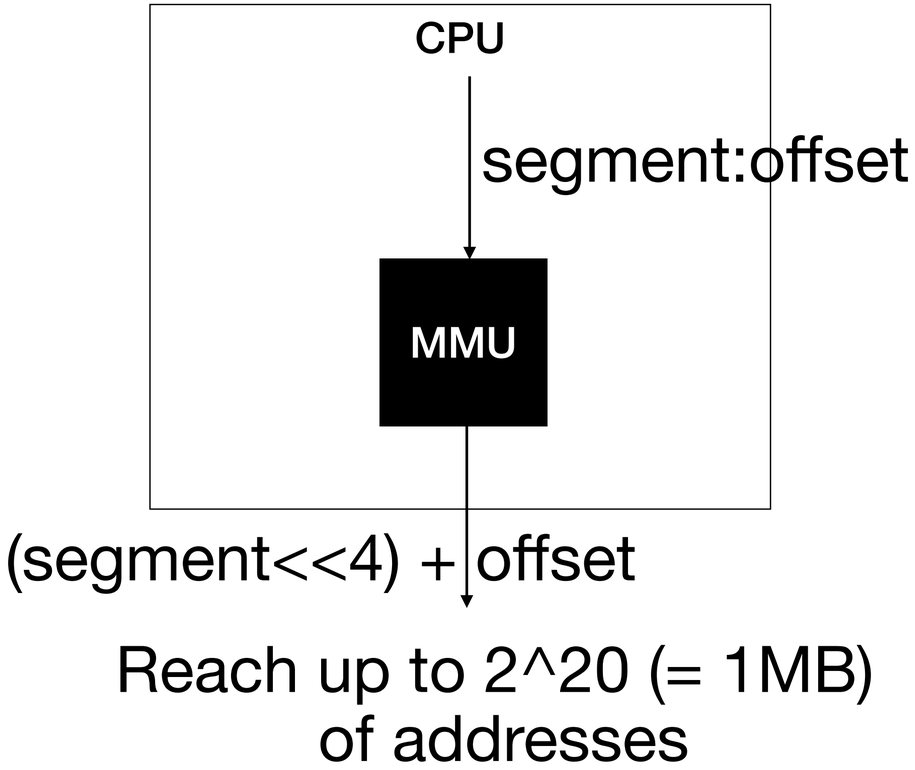
Segment registers

- 16-bit registers can only point up to 2^16 (=64KB) addresses in DRAM
- Full address = (segment registers : offset)
  - code segment (cs): ip



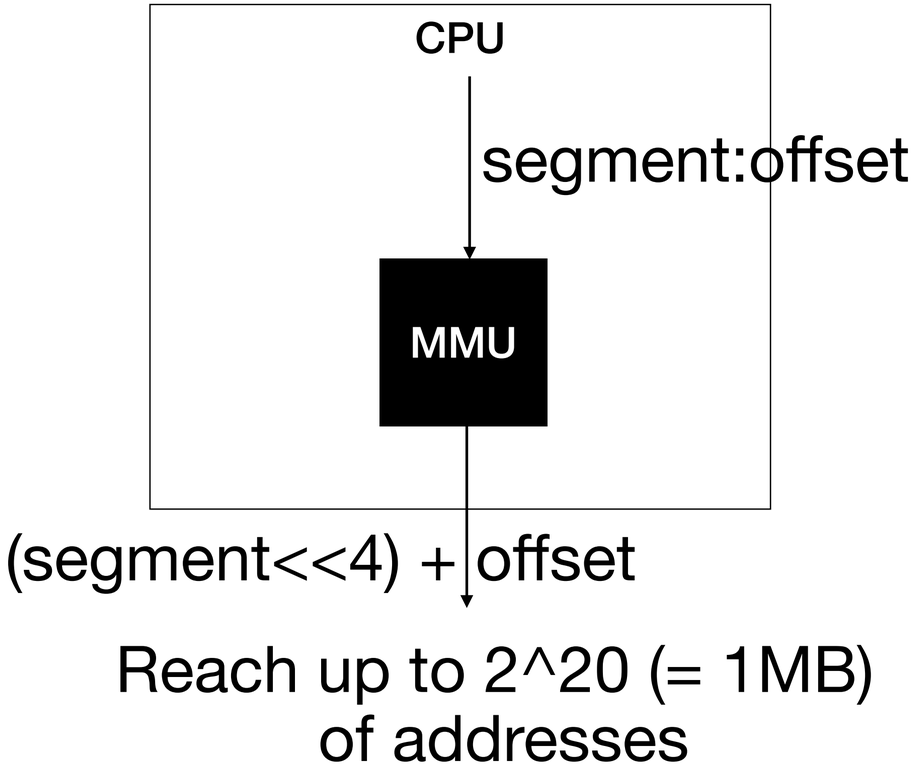
Segment registers

- 16-bit registers can only point up to 2^16 (=64KB) addresses in DRAM
- Full address = (segment registers : offset)
  - code segment (cs): ip
  - stack segment (ss): sp / bp.
    push, pop



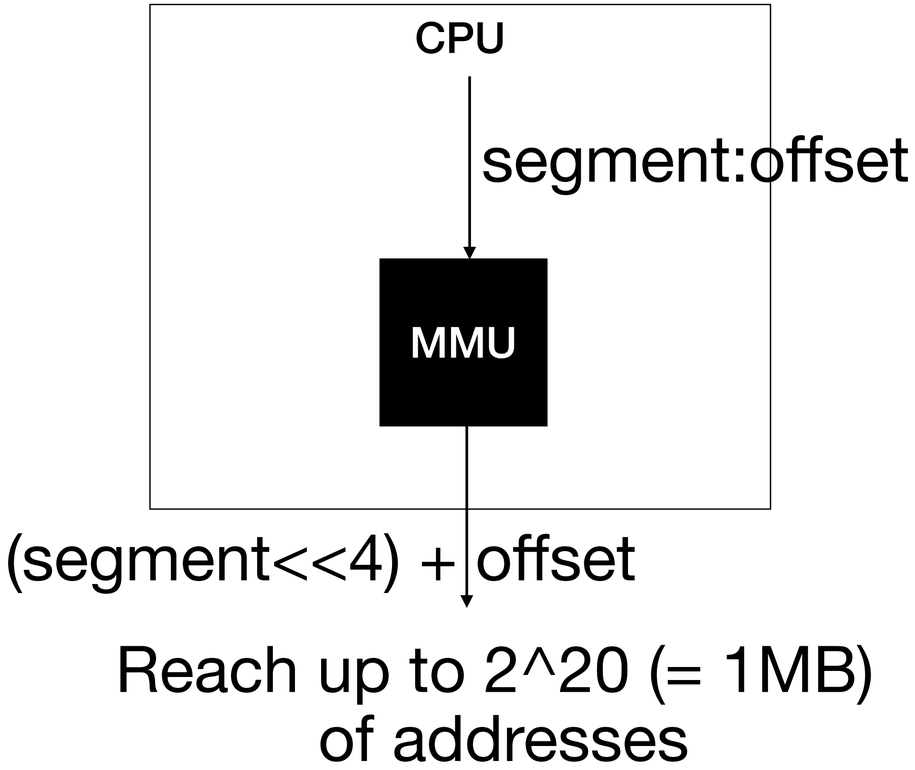
Segment registers

- 16-bit registers can only point up to 2^16 (=64KB) addresses in DRAM
- Full address = (segment registers : offset)
  - code segment (cs): ip
  - stack segment (ss): sp / bp.
    push, pop
  - data segment (ds): ax, bx, cx, dx.
    mov (%bx) %ax



Segment registers

- 16-bit registers can only point up to 2^16 (=64KB) addresses in DRAM
- Full address = (segment registers : offset)
  - code segment (cs): ip
  - stack segment (ss): sp / bp.
    push, pop
  - data segment (ds): ax, bx, cx, dx.
    mov (%bx) %ax
  - extra segment (es): si, di
    movsb



Far pointers in 16-bit x86

```
#include <stdio.h>
int foo() {
   char far *p =(char far *)0x55550005;
   char far *q =(char far *)0x53332225;
   *p = 80;
   (*p)++;
   printf("%d",*q);
   return 0;
}
```

Outputs 81

Far pointers in 16-bit x86

```
#include <stdio.h>
int foo() {
   char far *p =(char far *)0x55550005;
   char far *q =(char far *)0x53332225;
   *p = 80;
   (*p)++;
   printf("%d",*q);
   return 0;
}

Outputs 81
```

- Exposed to programmer via “far pointers”

Far pointers in 16-bit x86

```
#include <stdio.h>
int foo() {
   char far *p =(char far *)0x55550005;
   char far *q =(char far *)0x53332225;
   *p = 80;
   (*p)++;
   printf("%d",*q);
   return 0;
}
```

Outputs 81

- Exposed to programmer via “far pointers”
- p points to (0x5555 << 4) + 0x0005 = 0x55555

Far pointers in 16-bit x86

```
#include <stdio.h>
int foo() {
   char far *p =(char far *)0x55550005;
   char far *q =(char far *)0x53332225;
   *p = 80;
   (*p)++;
   printf("%d",*q);
   return 0;
}
```

Outputs 81

- Exposed to programmer via “far pointers”
- p points to (0x5555 << 4) + 0x0005 = 0x55555
- q points to (0x5333 << 4) + 0x2225 = 0x55555

Far pointers in 16-bit x86

```c
#include <stdio.h>
int foo() {
   char far *p =(char far *)0x55550005;
   char far *q =(char far *)0x53332225;
   *p = 80;
   (*p)++;
   printf("%d",*q);
   return 0;
}
```

Outputs 81

- Exposed to programmer via “far pointers”
- p points to (0x5555 << 4) + 0x0005 = 0x55555
- q points to (0x5333 << 4) + 0x2225 = 0x55555
- Multiple ways of referencing same address making them awkward to control

Far pointers in 16-bit x86

```c
#include <stdio.h>
int foo() {
   char far *p =(char far *)0x55550005;
   char far *q =(char far *)0x53332225;
   *p = 80;
   (*p)++;
   printf("%d",*q);
   return 0;
}
```

Outputs 81

- Exposed to programmer via “far pointers”
- p points to (0x5555 << 4) + 0x0005 = 0x55555
- q points to (0x5333 << 4) + 0x2225 = 0x55555
- Multiple ways of referencing same address making them awkward to control

Segment registers in 32-bit

*: S.No. added only for illustration

Segment registers in 32-bit

- 32-bit registers can point to 2^32 (=4GB) memory.

*: S.No. added only for illustration

Segment registers in 32-bit

- 32-bit registers can point to 2^32 (=4GB) memory.
- “Protected mode”: extend segment registers for *protection*

*: S.No. added only for illustration

Segment registers in 32-bit

- 32-bit registers can point to 2^32 (=4GB) memory.
- “Protected mode”: extend segment registers for *protection*

GDTR: Global descriptor table register

cs: top 13 bits
0b…10xxx

Global Descriptor Table

S.No.*	Permission	Base	Limit
0x01	WRITE		
0x02	EXECUTE	0x1F	0xFF
…	…	…	…

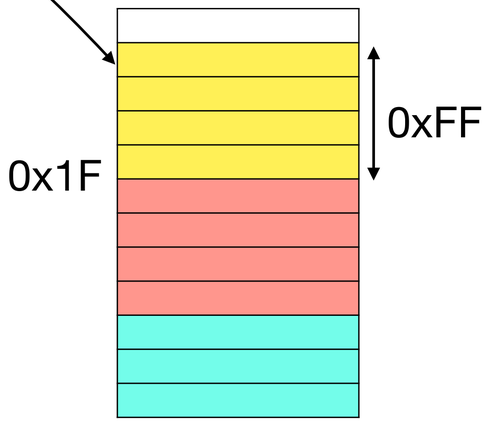
*: S.No. added only for illustration

Address translation

Global Descriptor Table

cs: top 13 bits
0b…10xxx

S.No.*	Permission	Base	Limit
0x01	WRITE		
0x02	EXECUTE	0x1F	0xFF
…	…	…	…

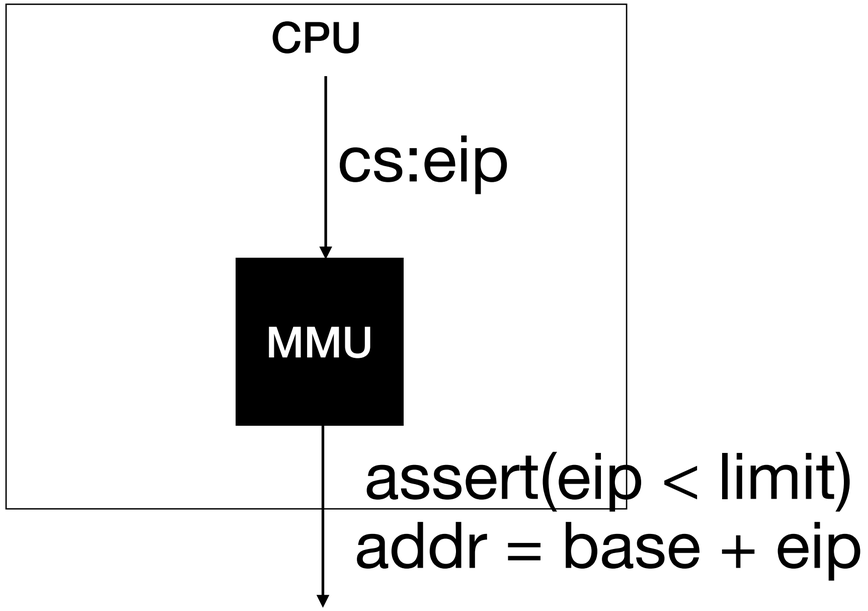


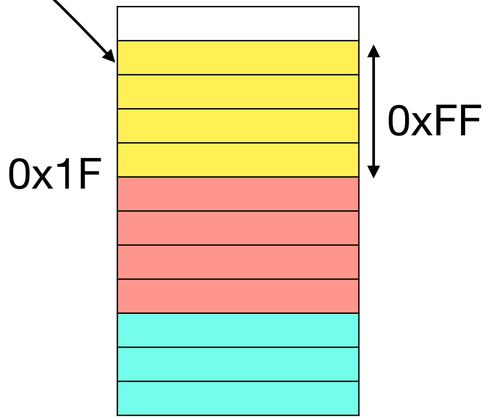
Address translation

cs: top 13 bits
0b…10xxx

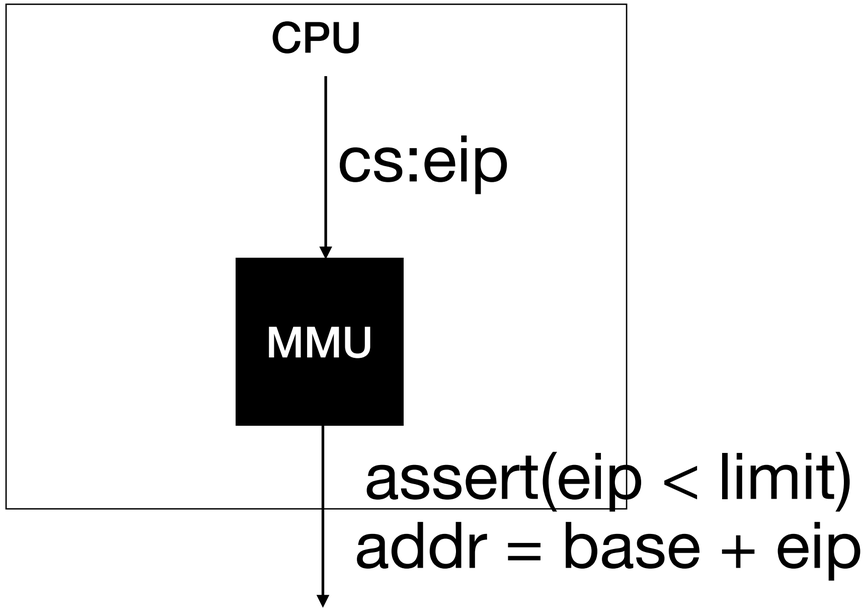
Global Descriptor Table

S.No.*	Permission	Base	Limit
0x01	WRITE		
0x02	EXECUTE	0x1F	0xFF
…	…	…	…





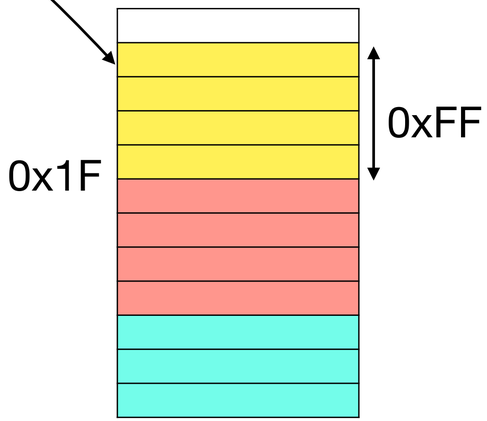
Address translation



cs: top 13 bits
0b…10xxx

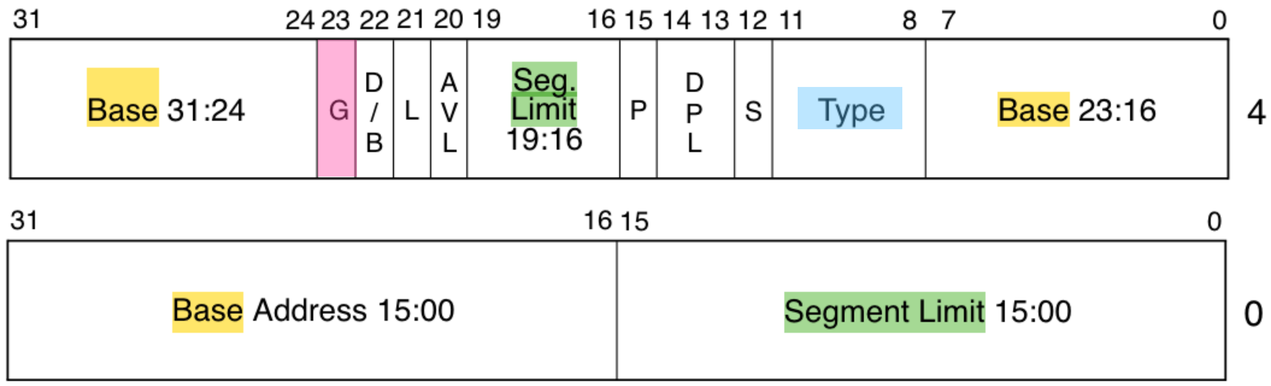
Global Descriptor Table

S.No.*	Permission	Base	Limit
0x01	WRITE		
0x02	EXECUTE	0x1F	0xFF
…	…	…	…



- Can “protect” different segments from each other

Global descriptor table

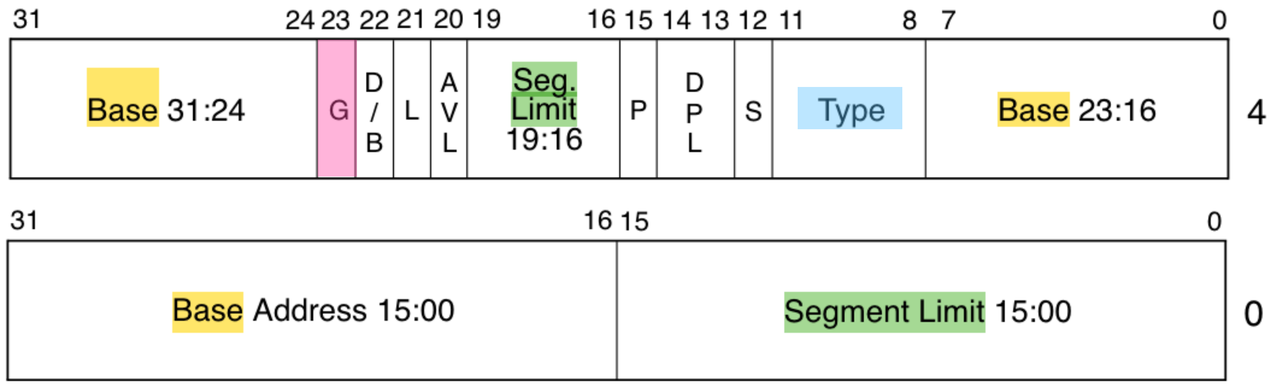


L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

Global descriptor table

- Upto 2^13 (=8192) segment descriptors

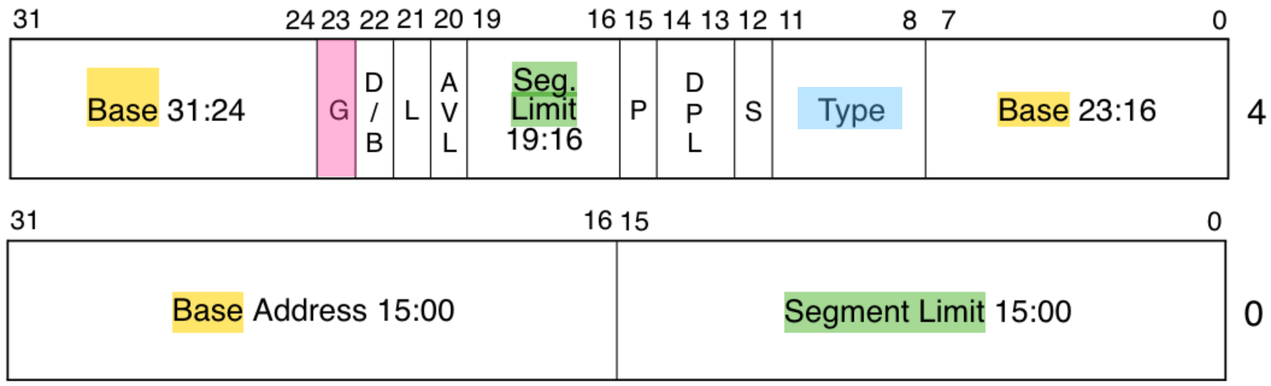


L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

Global descriptor table

- Upto 2^13 (=8192) segment descriptors
- Segment descriptor:

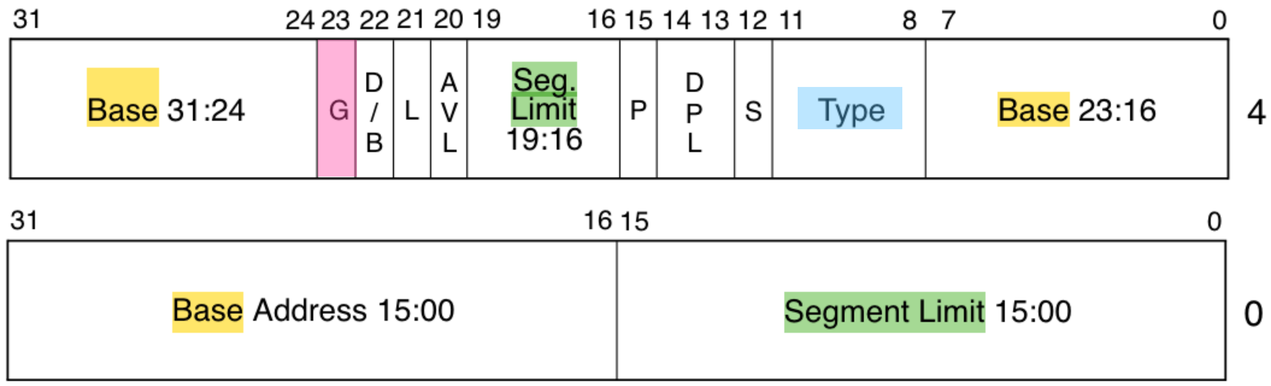


L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

Global descriptor table

- Upto 2^13 (=8192) segment descriptors
- Segment descriptor:
  - 32 bit base

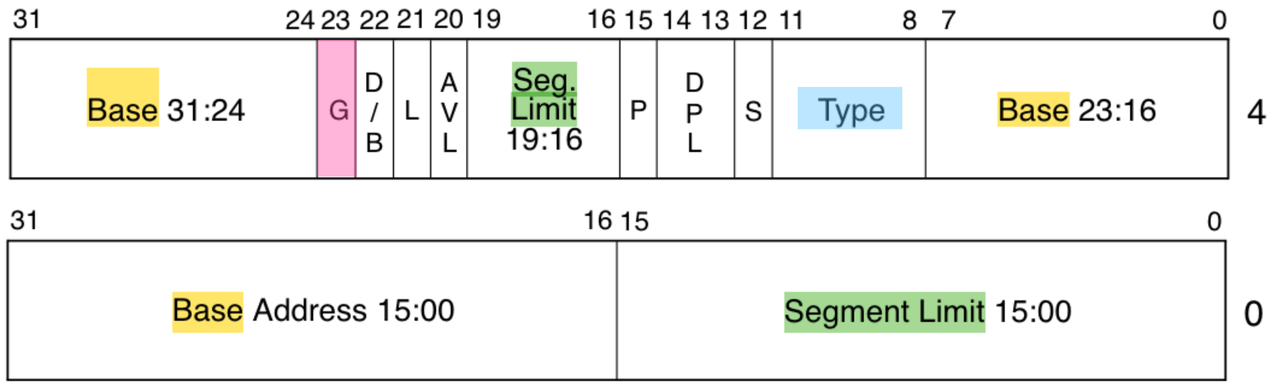


L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

Global descriptor table

- Upto 2^13 (=8192) segment descriptors
- Segment descriptor:
  - 32 bit base
  - 20 bit limit

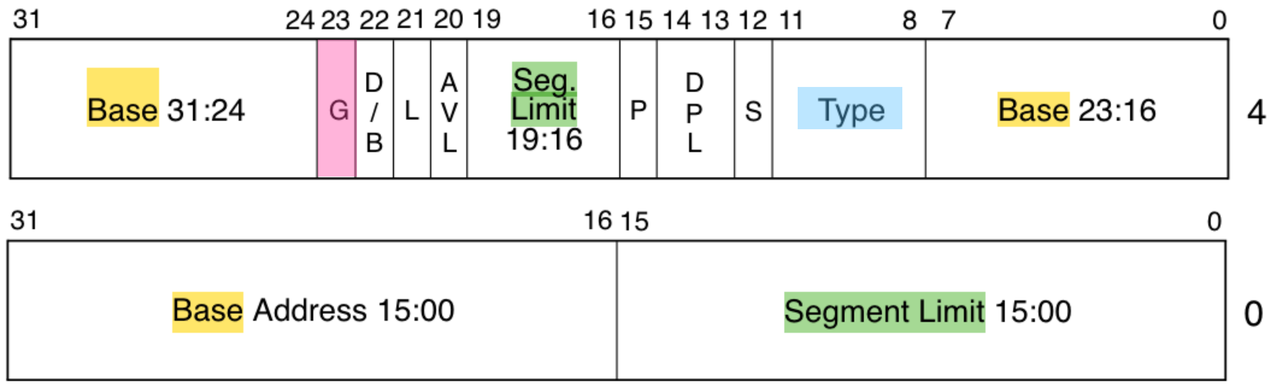


L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

Global descriptor table

- Upto 2^13 (=8192) segment descriptors
- Segment descriptor:
  - 32 bit base
  - 20 bit limit
  - If G=1, granularity=4KB (2^12).

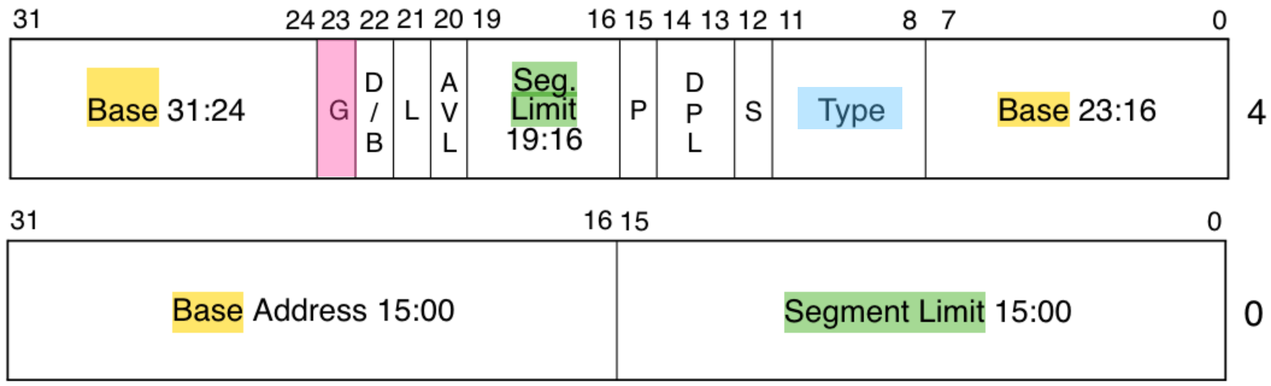


L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

Global descriptor table

- Upto 2^13 (=8192) segment descriptors
- Segment descriptor:
  - 32 bit base
  - 20 bit limit
  - If G=1, granularity=4KB (2^12).
  - Max memory in a segment = 2^20*2^12 = 4GB



L — 64-bit code segment (IA-32e mode only)
AVL — Available for use by system software
BASE — Segment base address
D/B — Default operation size (0 = 16-bit segment; 1 = 32-bit segment)
DPL — Descriptor privilege level
G — Granularity
LIMIT — Segment Limit
P — Segment present
S — Descriptor type (0 = system; 1 = code or data)
TYPE — Segment type

Figure 3-8. Segment Descriptor

Segment descriptor type

- Code segment: 1
- Data segment: 0

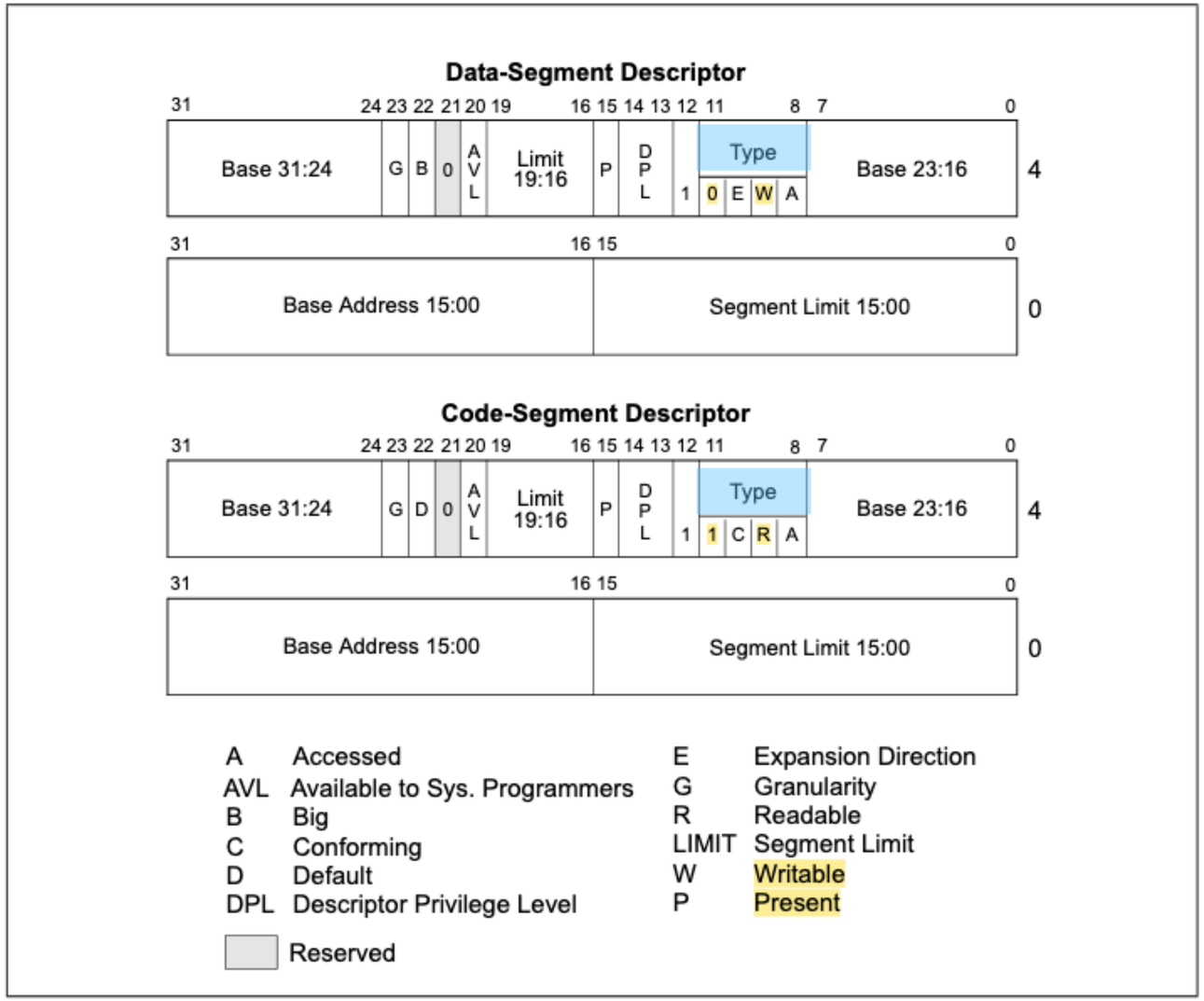


Figure 5-1. Descriptor Fields Used for Protection

Segmented memory model

- Stack cannot grow into code section

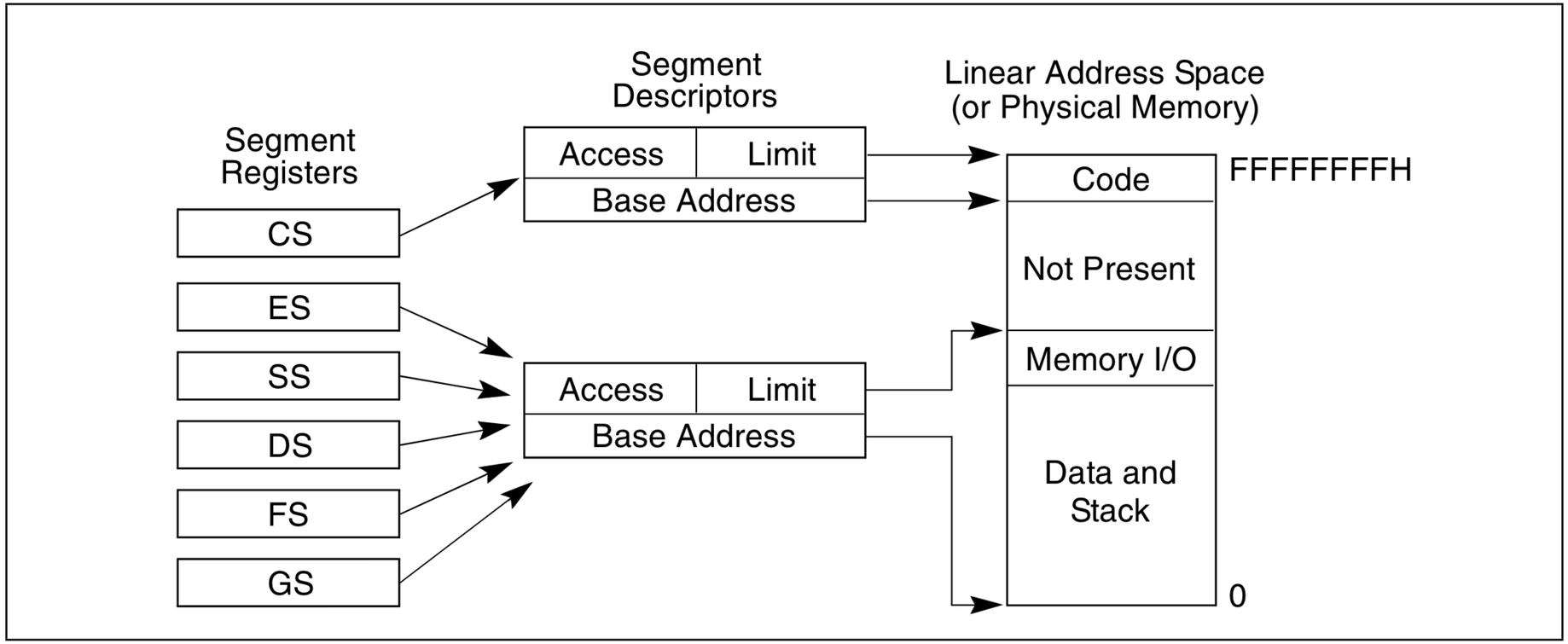


Figure 3-3. Protected Flat Model

Multi-segment model

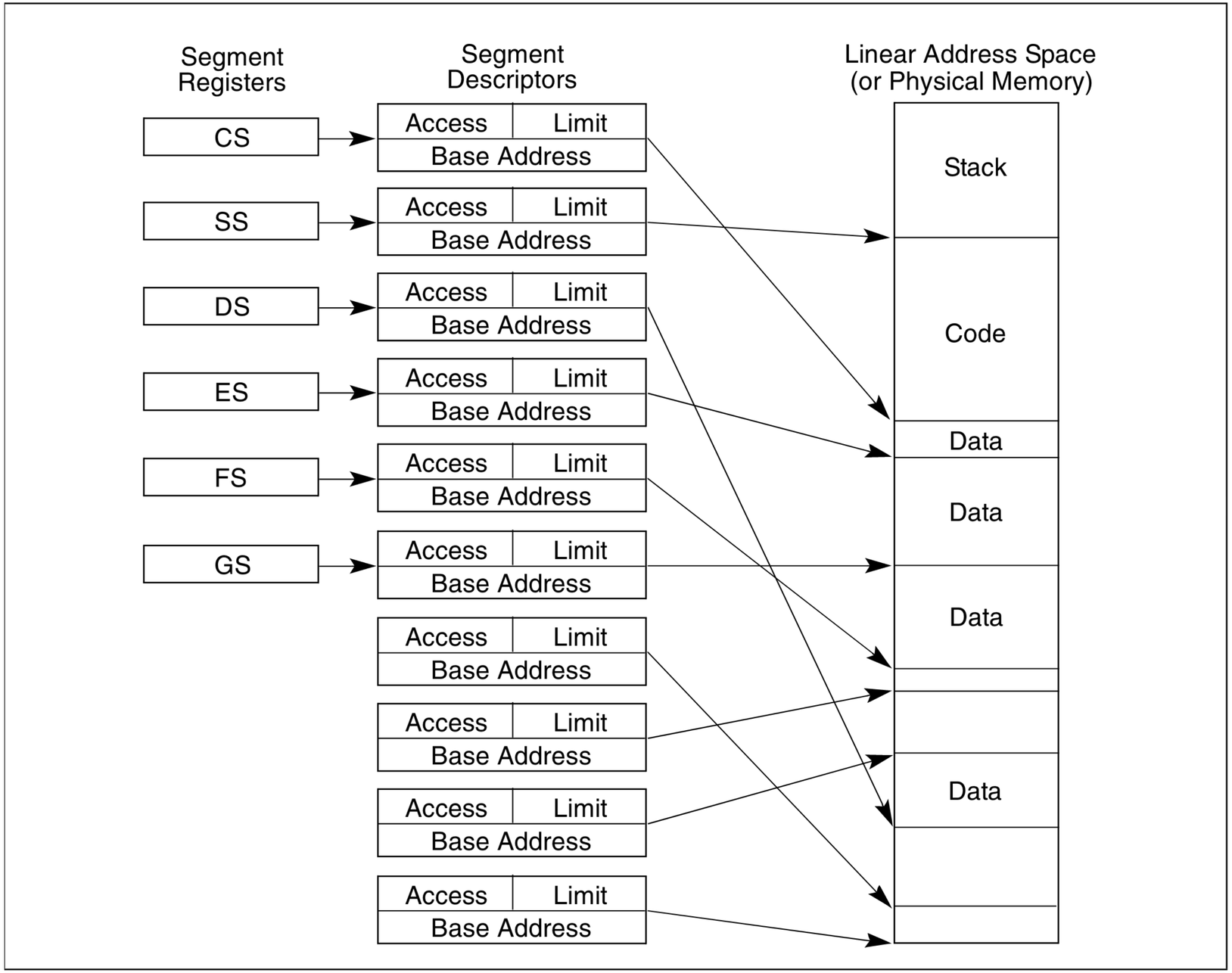


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection

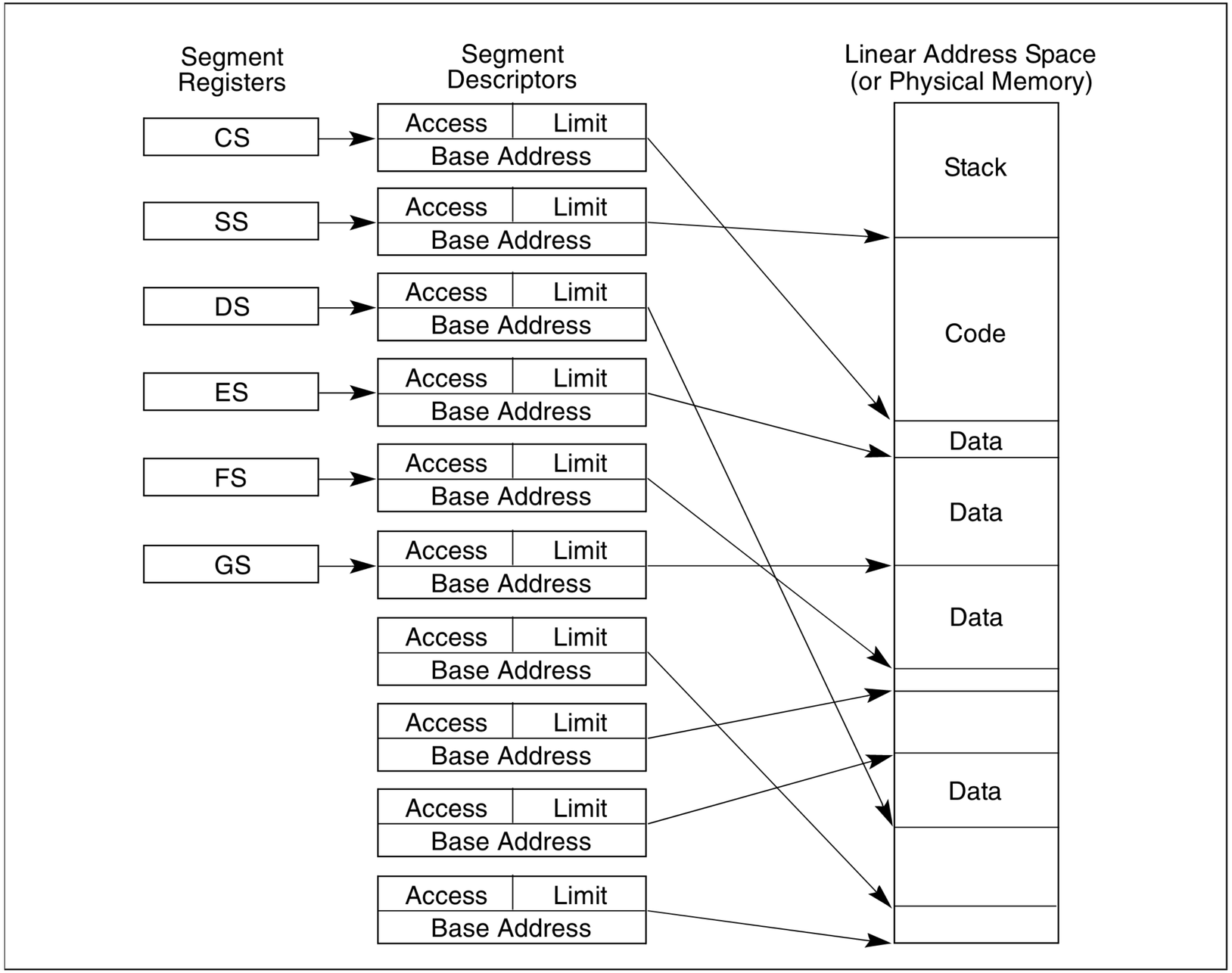


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection
- Difficult to program

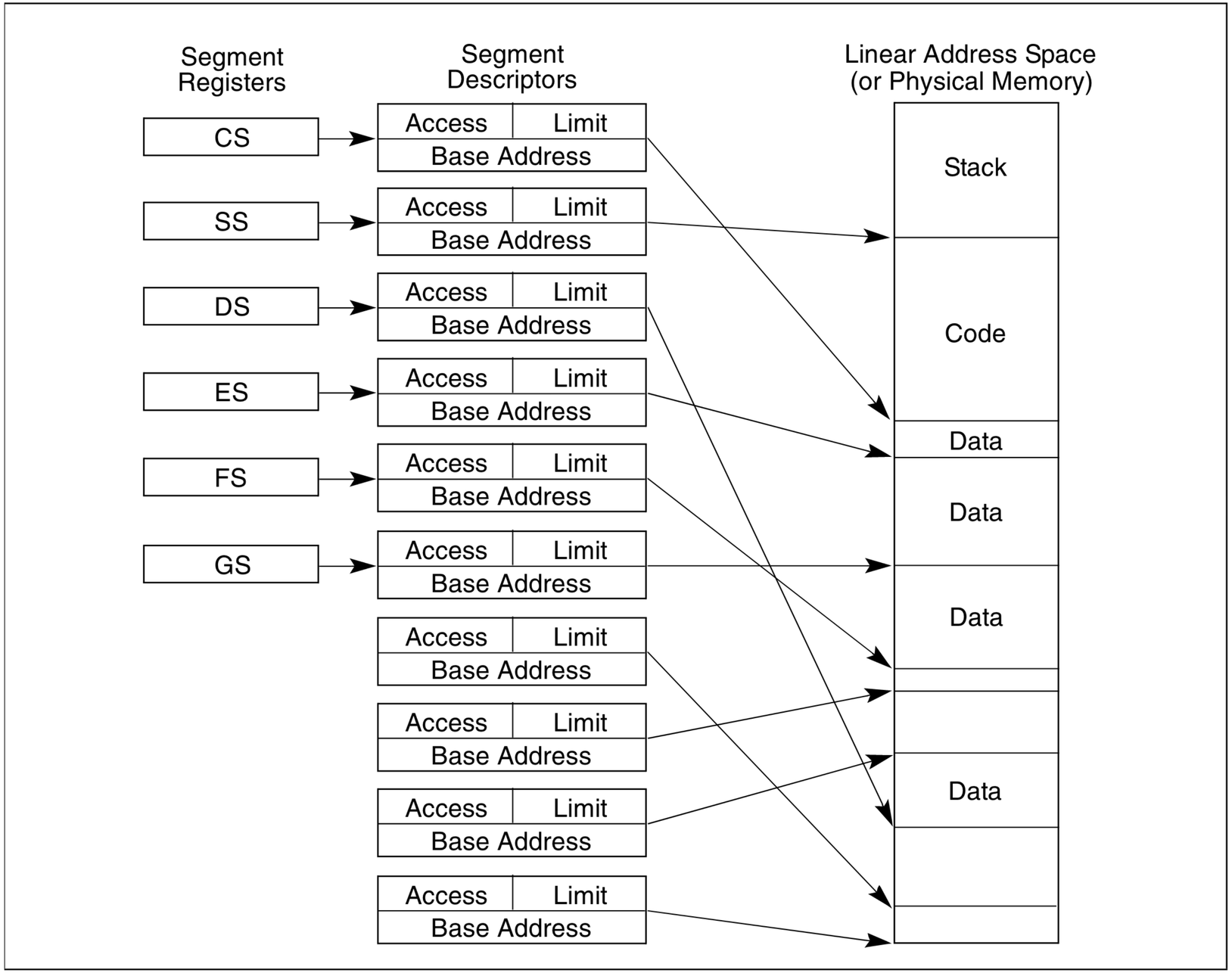


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection
- Difficult to program

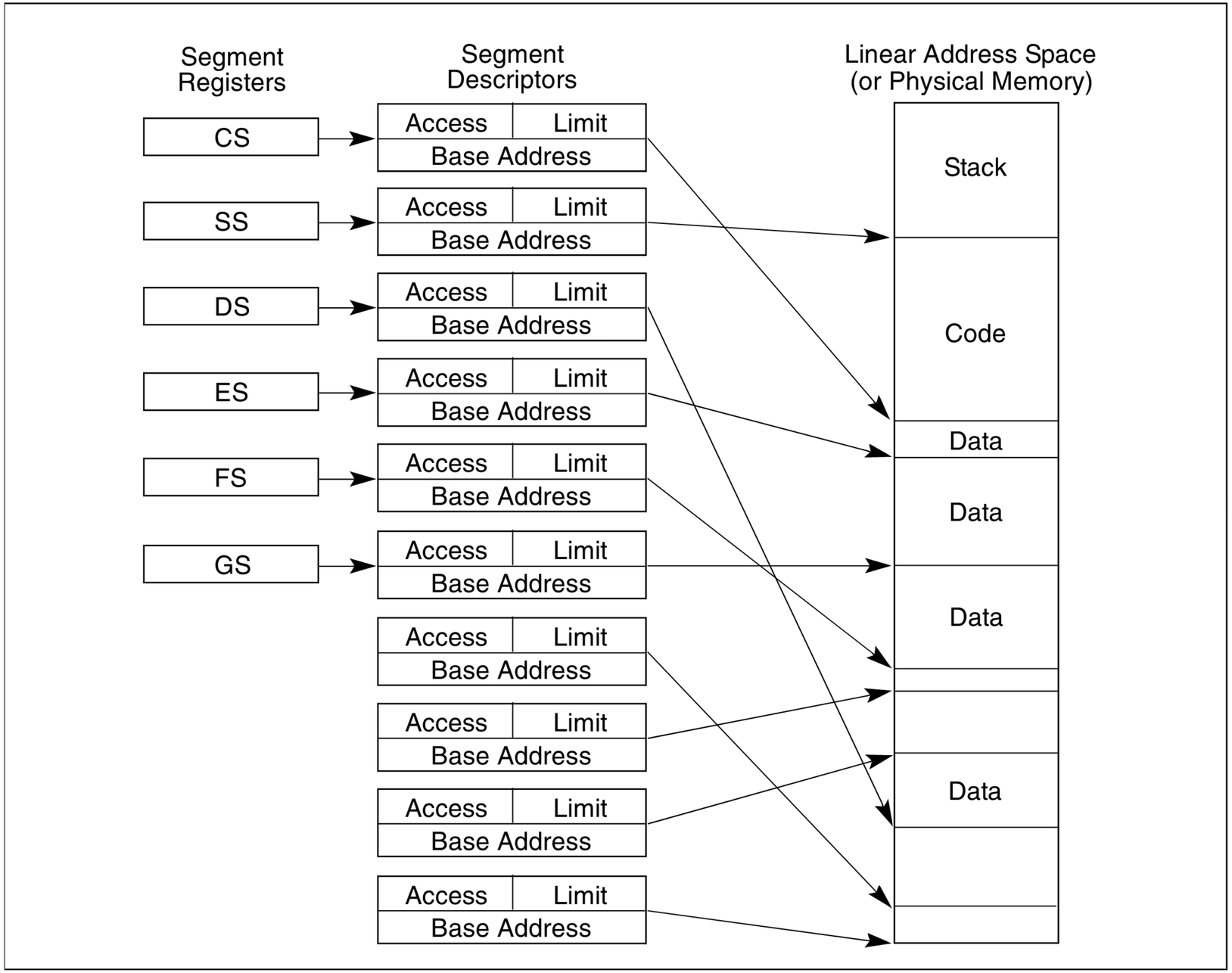


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection
- Difficult to program

movl %esp %ecx

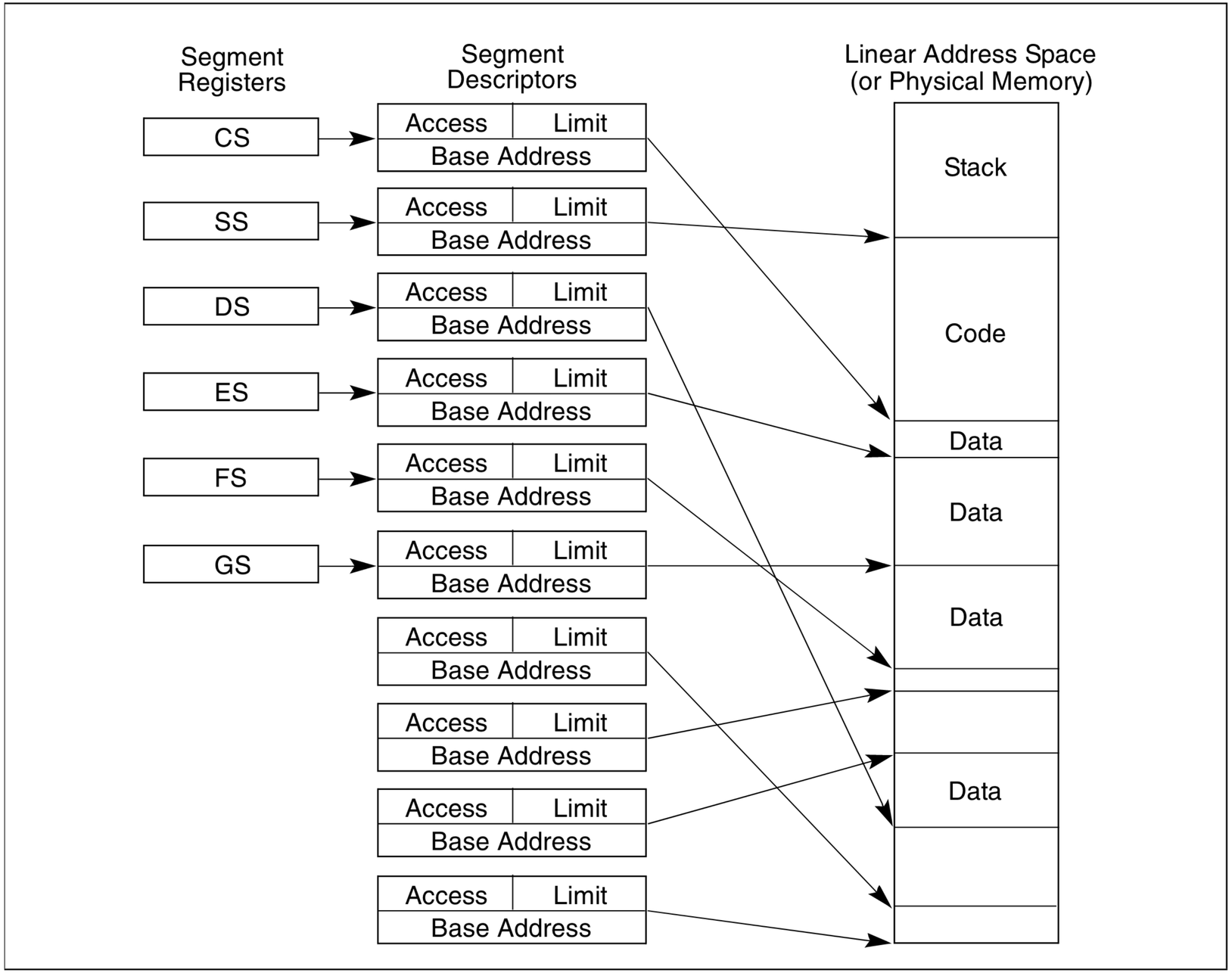


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection
- Difficult to program

movl %esp %ecx

addl $1 (%ecx)

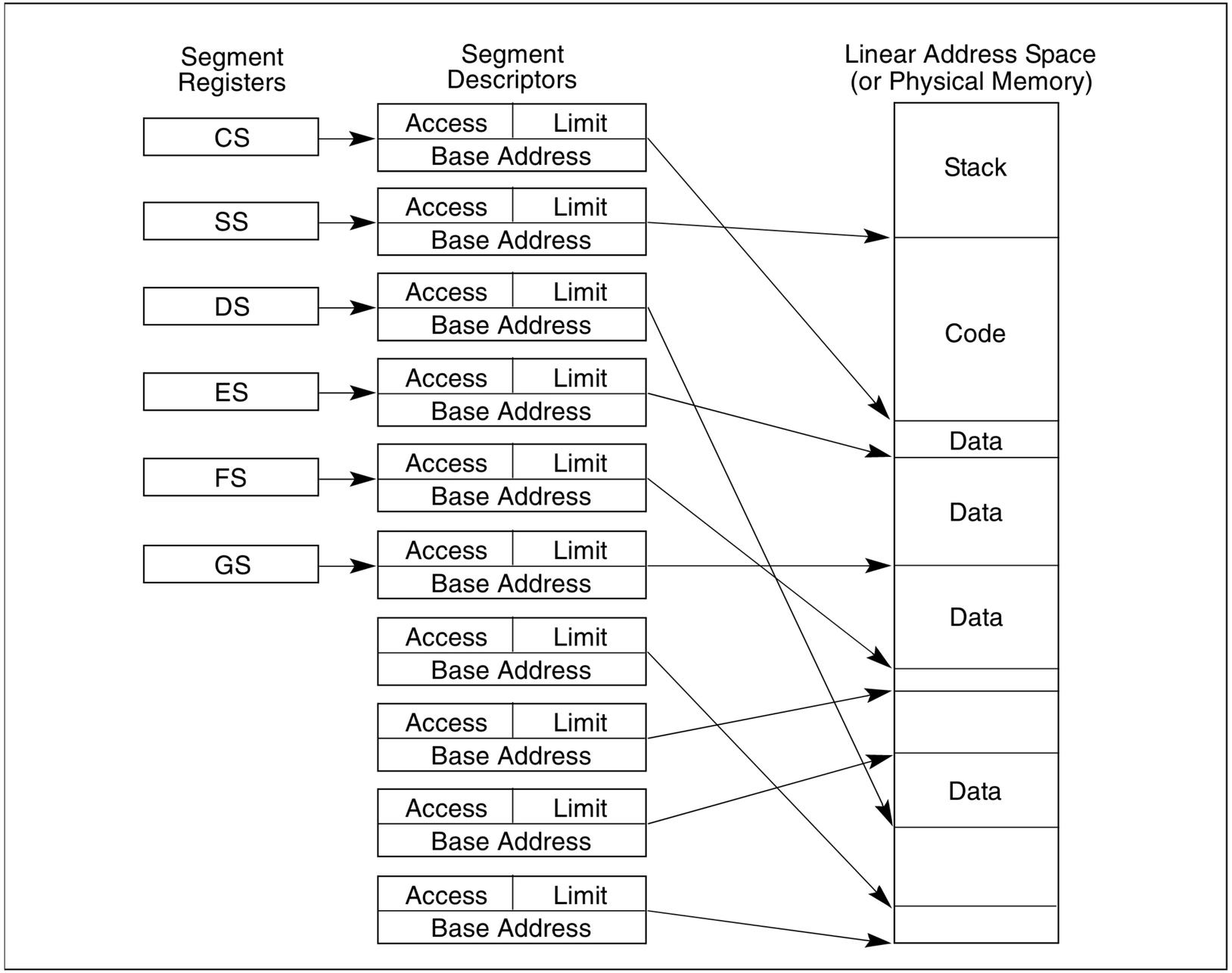


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection
- Difficult to program

movl %esp %ecx

addl $1 (%ecx)

Does not add 1 to the value at top of the stack!

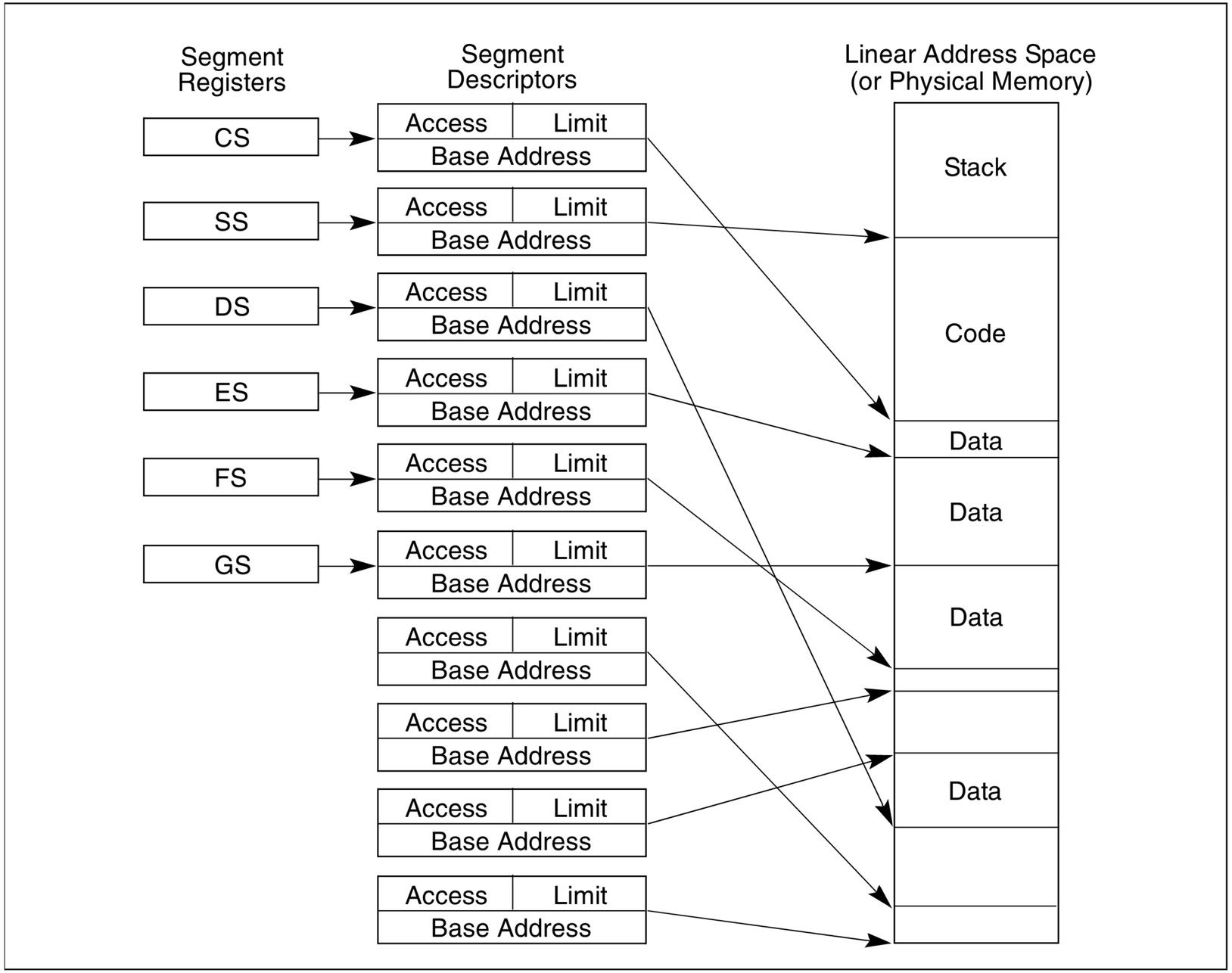


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection
- Difficult to program

movl %esp %ecx

addl $1 (%ecx)

Does not add 1 to the value at top of the stack!

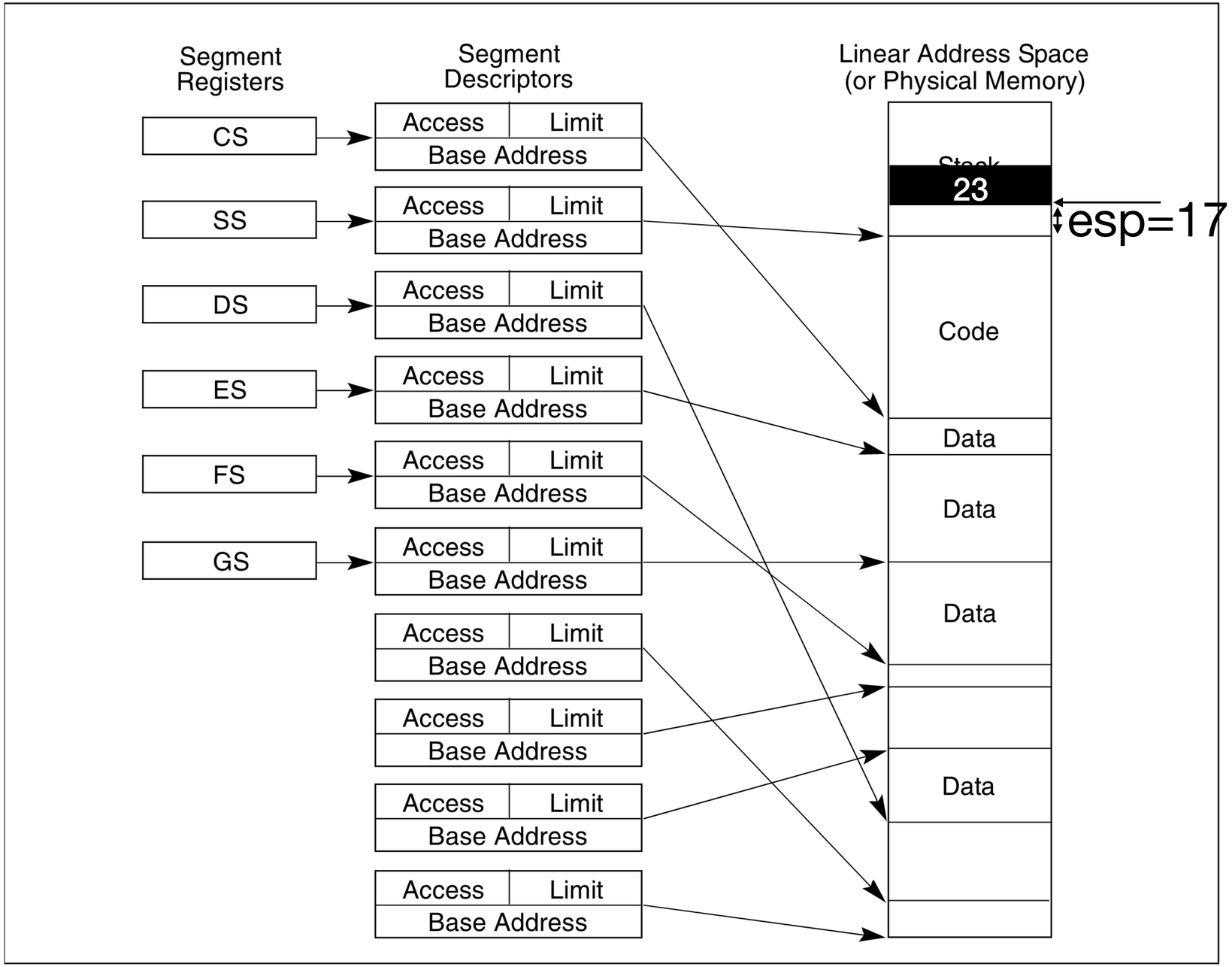


Figure 3-4. Multi-Segment Model

Multi-segment model

- Best protection
- Difficult to program

movl %esp %ecx

addl $1 (%ecx)

Does not add 1 to the value at top of the stack!

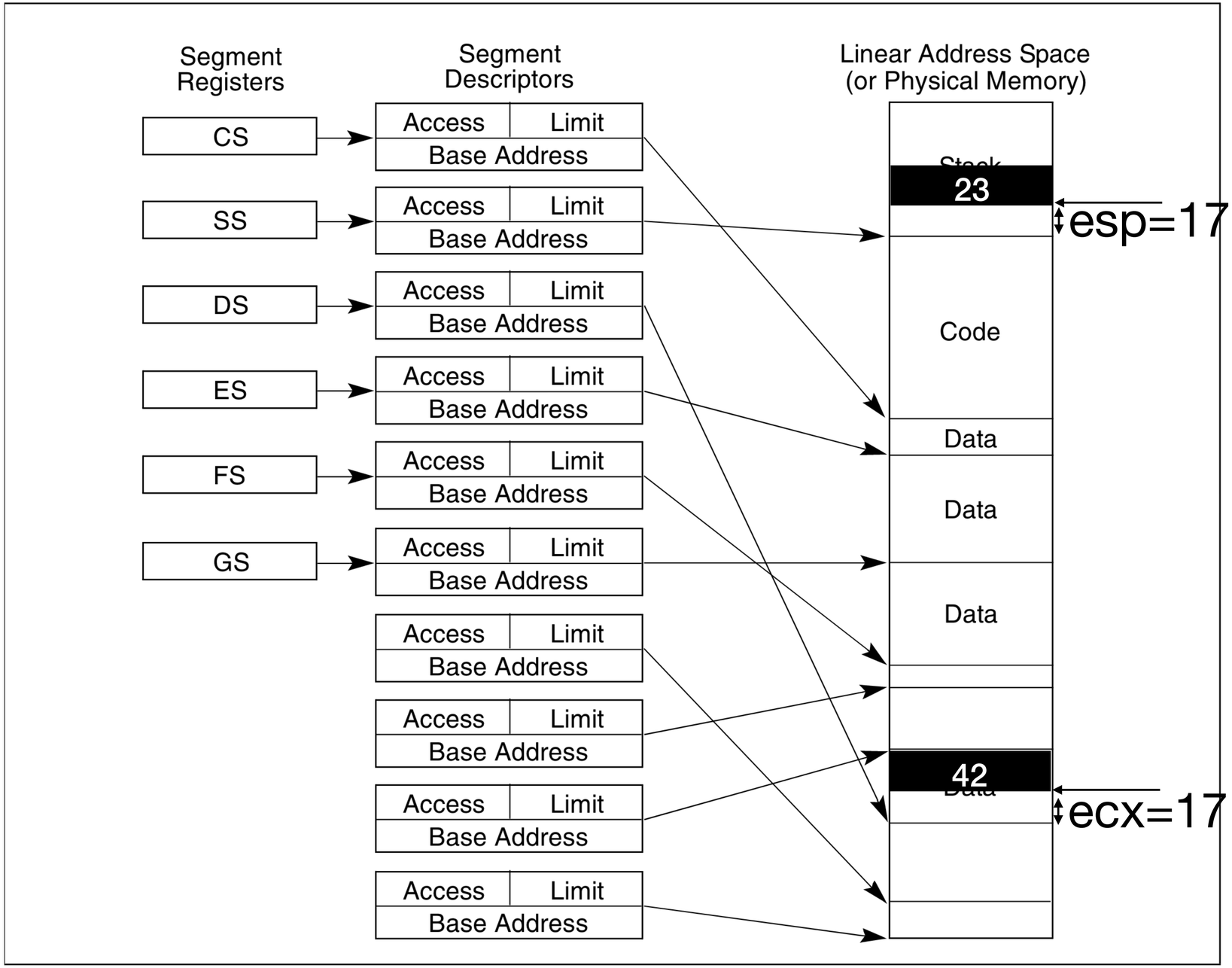


Figure 3-4. Multi-Segment Model

Flat memory model

- Easier to program
- Used by xv6

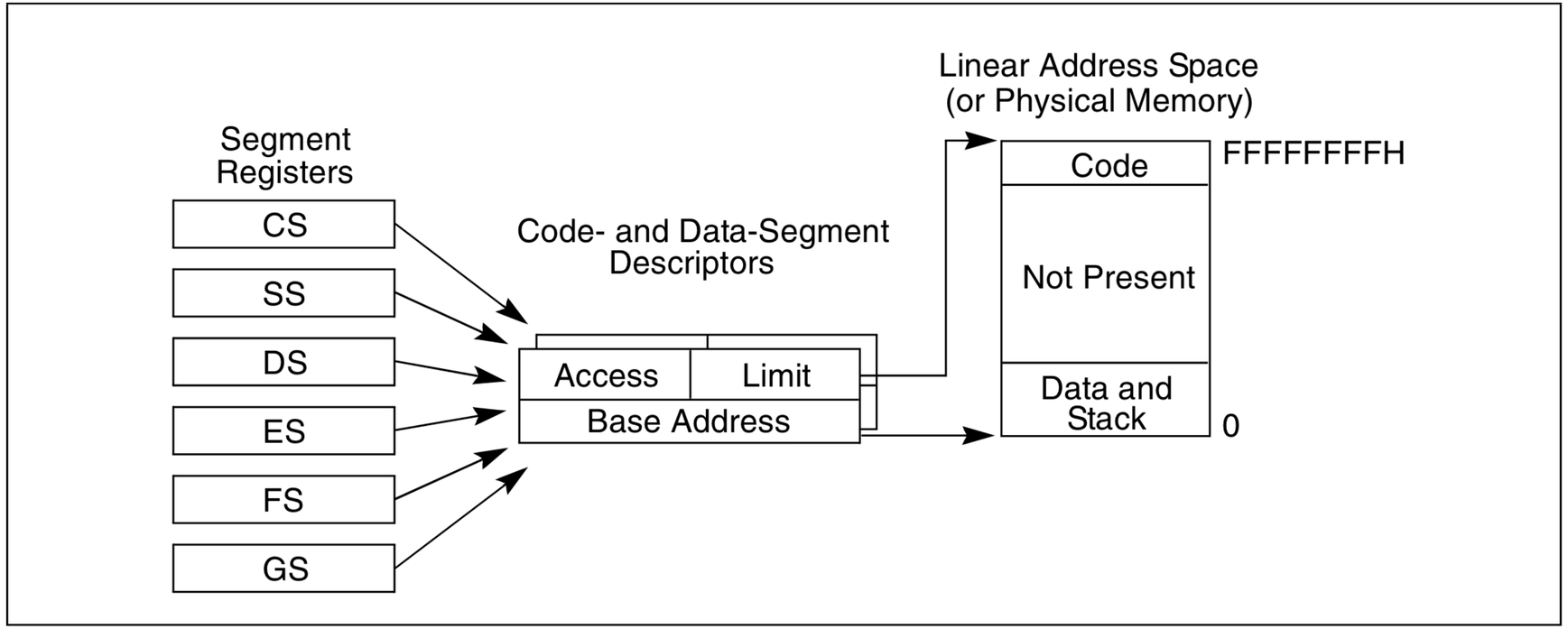
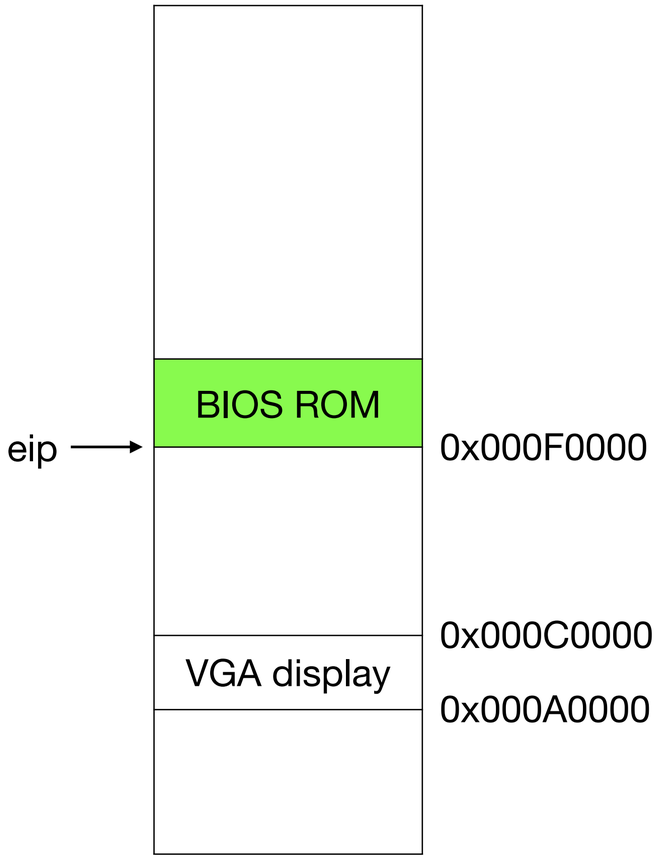


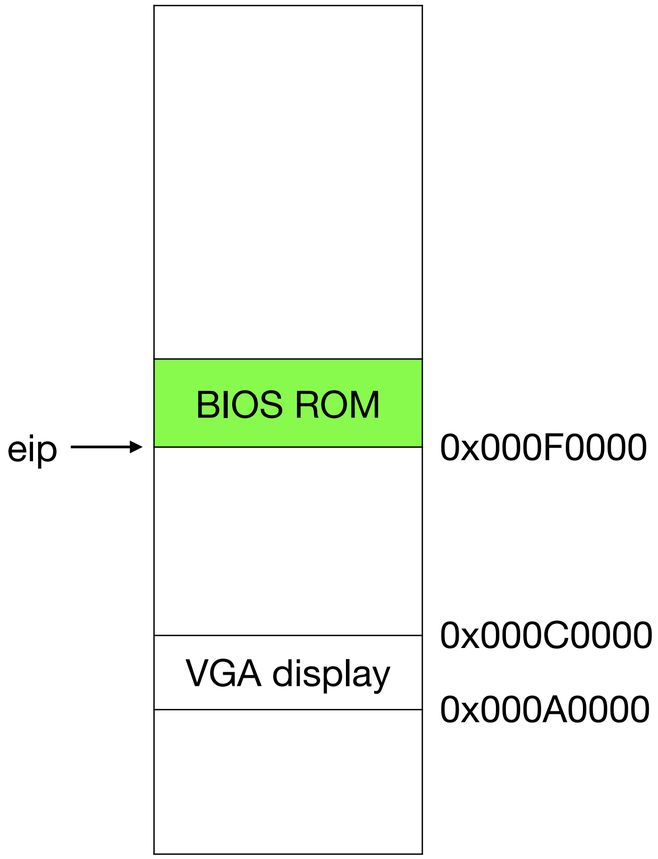
Figure 3-2. Flat Model

xv6 boot in action

Boot up sequence (1): BIOS

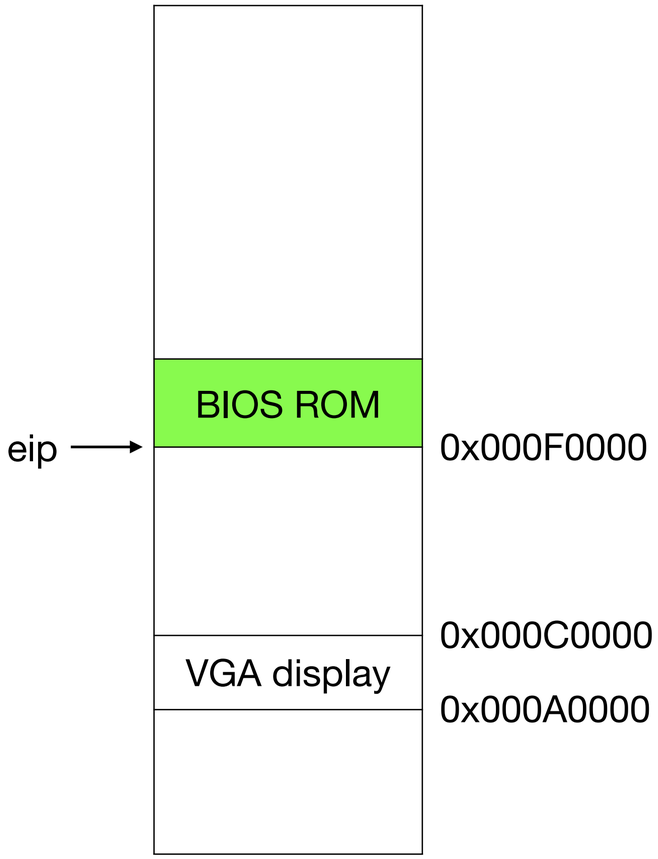


Boot up sequence (1): BIOS



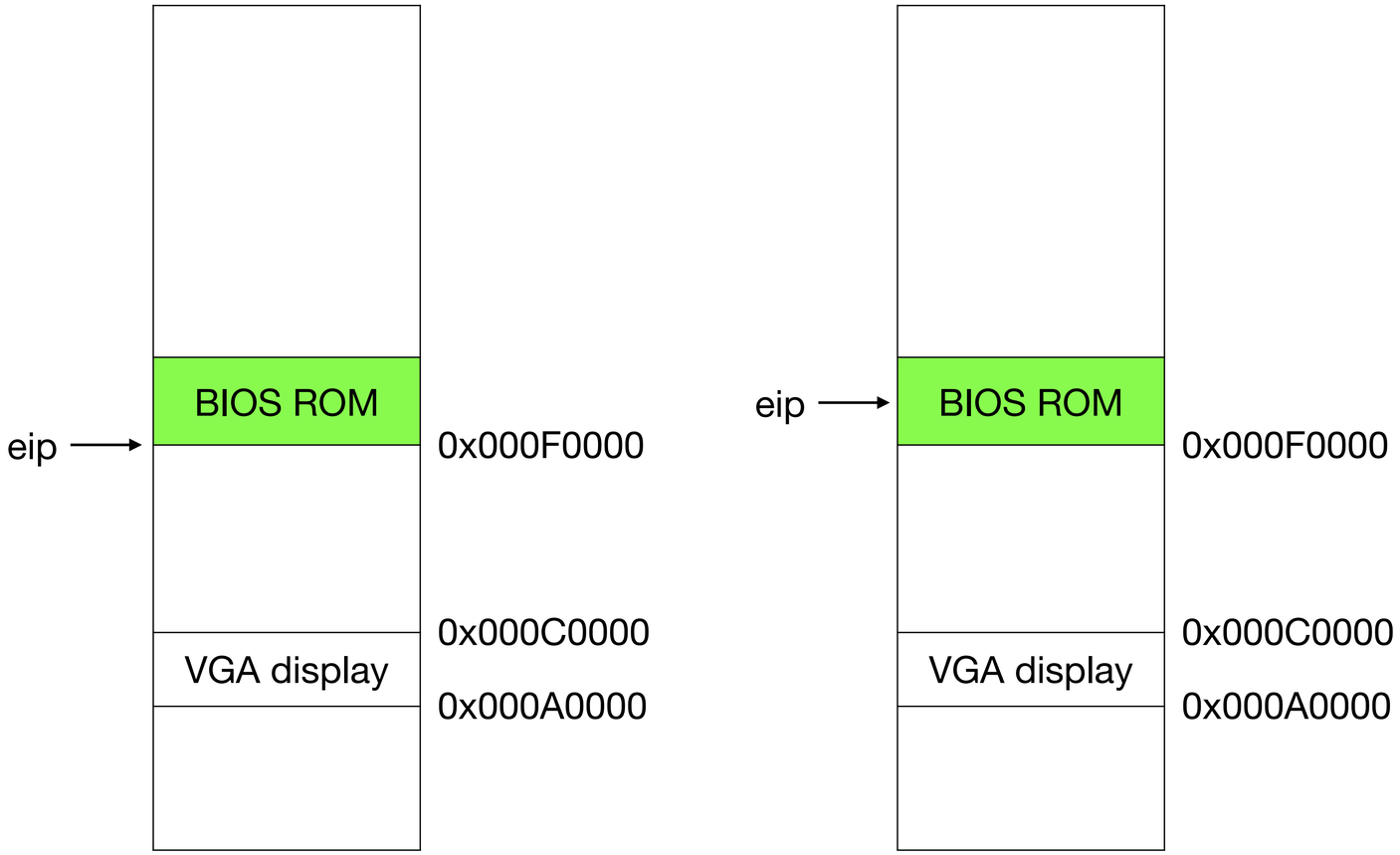
- For backward compatibility, PC boots in 16-bit mode

Boot up sequence (1): BIOS



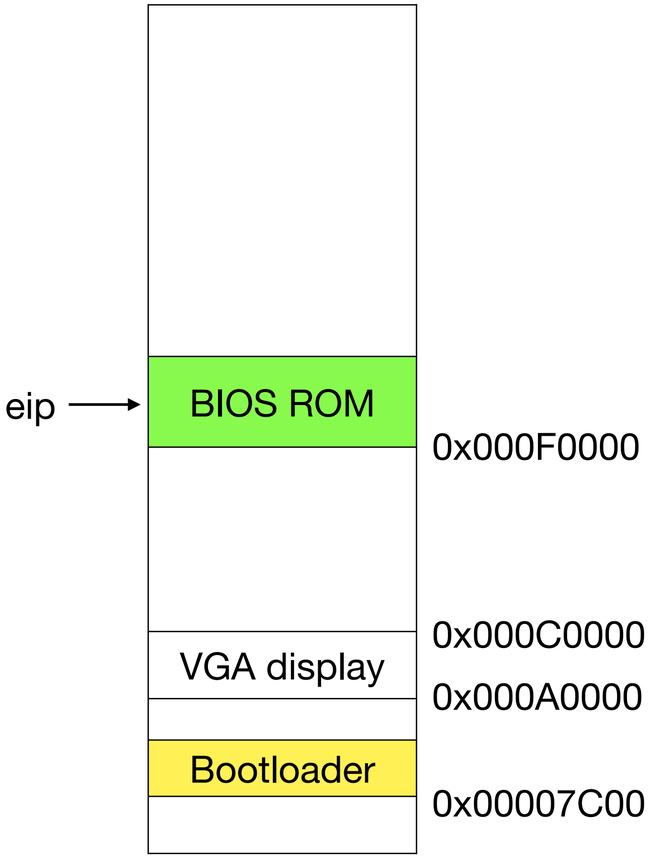
- For backward compatibility, PC boots in 16-bit mode
- BIOS does initial hardware check: are CPUs, memory, disk functional?

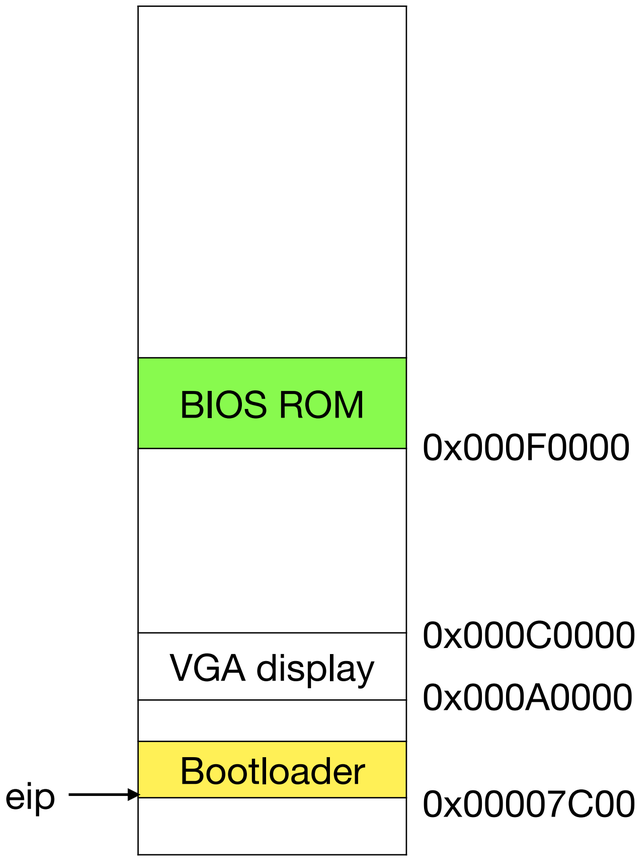
Boot up sequence (1): BIOS



- For backward compatibility, PC boots in 16-bit mode
- BIOS does initial hardware check: are CPUs, memory, disk functional?

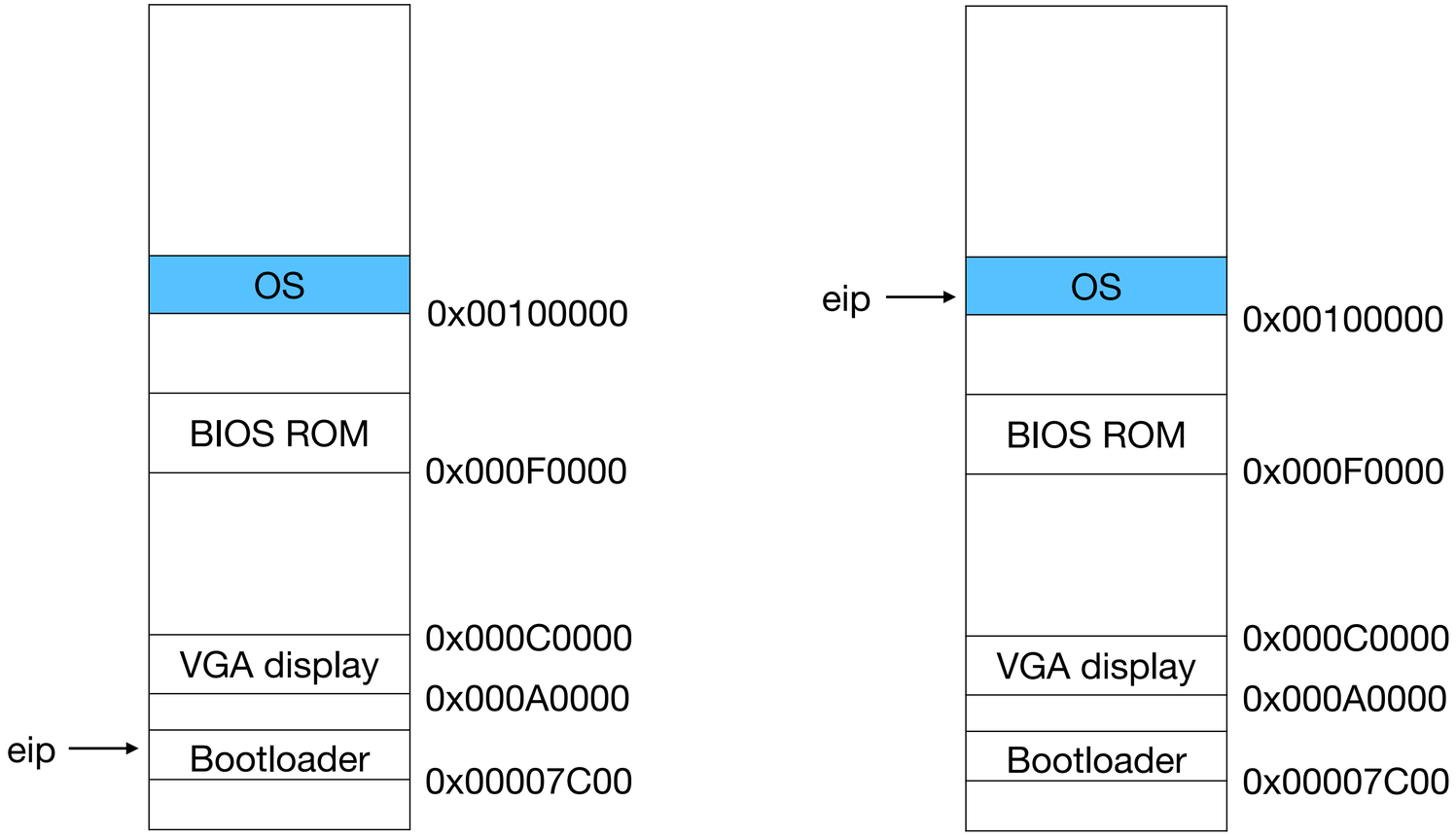
Boot up sequence (2): Bootloader



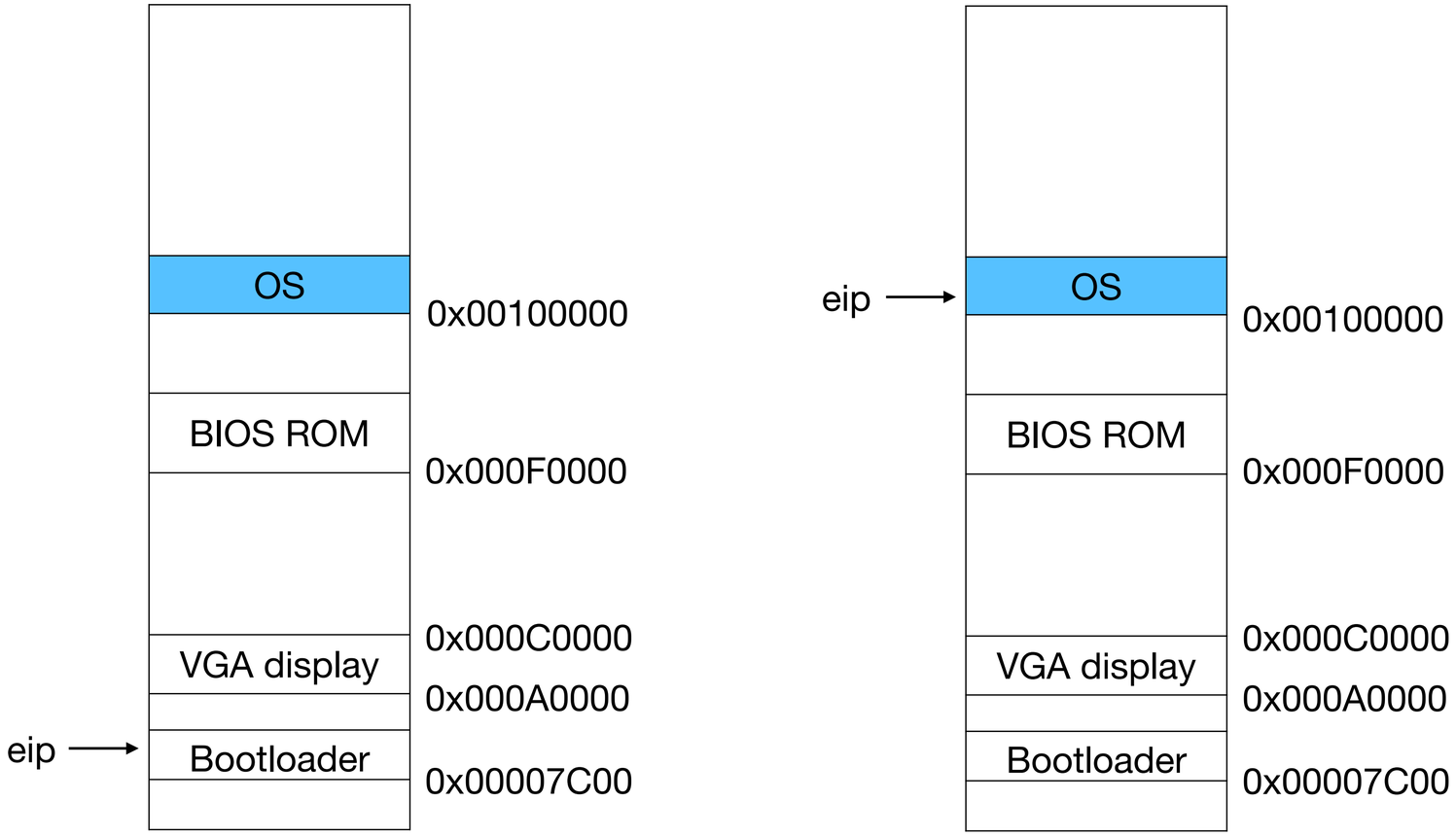


- BIOS loads first disk sector (512 bytes) at 0x7c00 and gives control
  - We need to write a boot loader that fits in the first sector of disk
  - We need to tell gcc that boot loader will load at 0x7c00 (virtual memory to physical memory translation is not setup yet!)
- Boot loader changes to 32 bit mode

Boot up sequence (3): OS

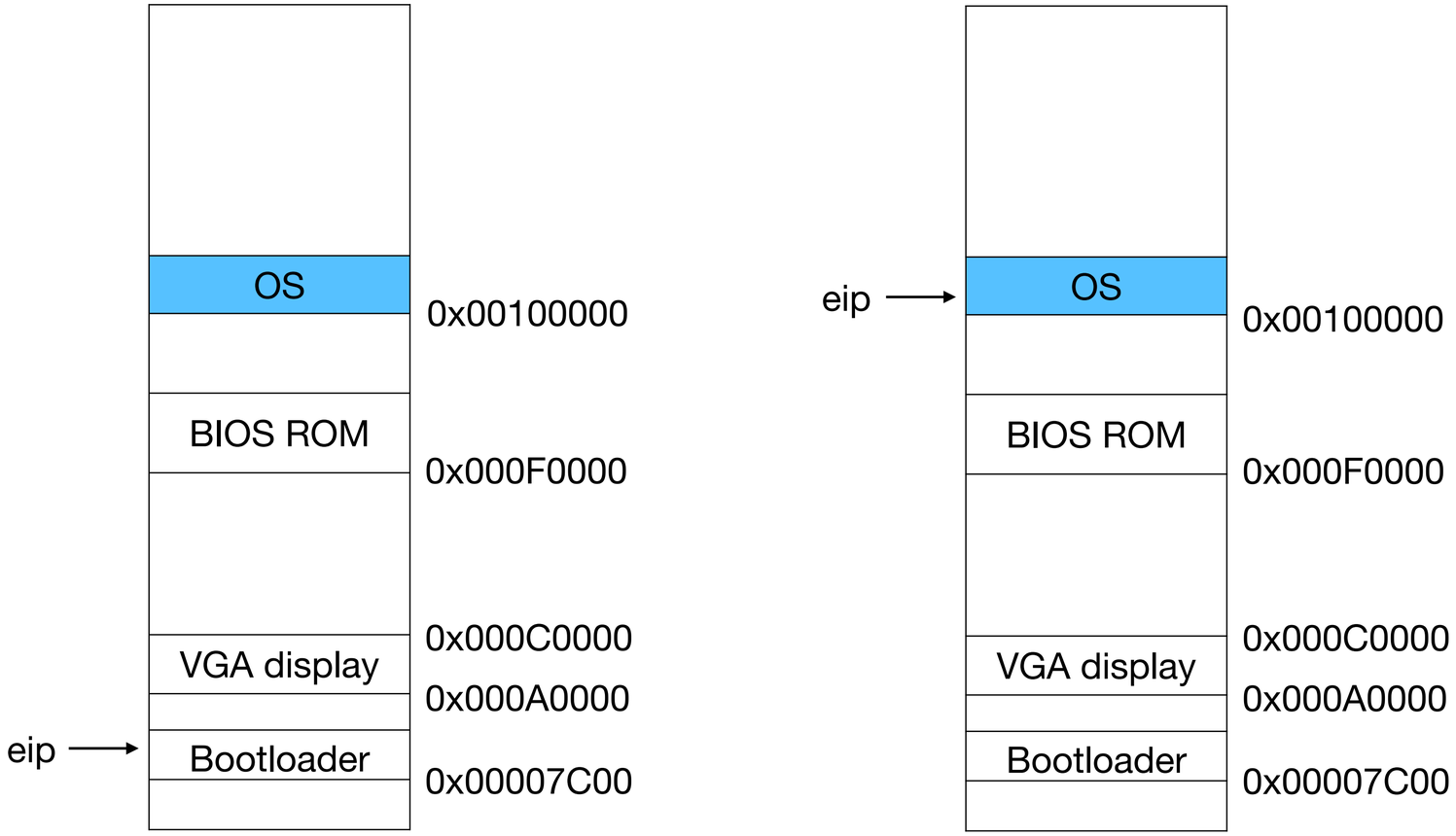


Boot up sequence (3): OS



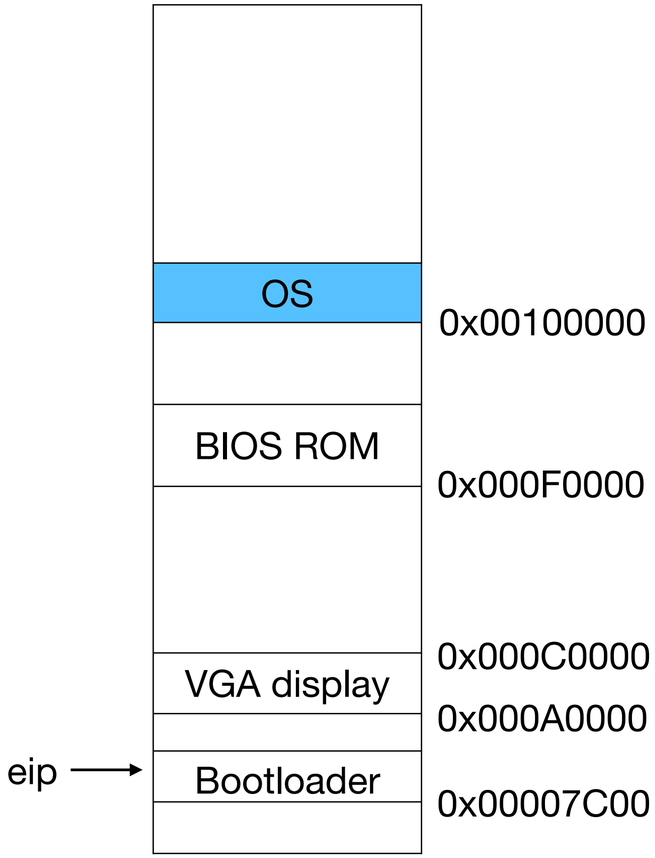
- gcc prepares OS image in Executable and Linkable Format (ELF)

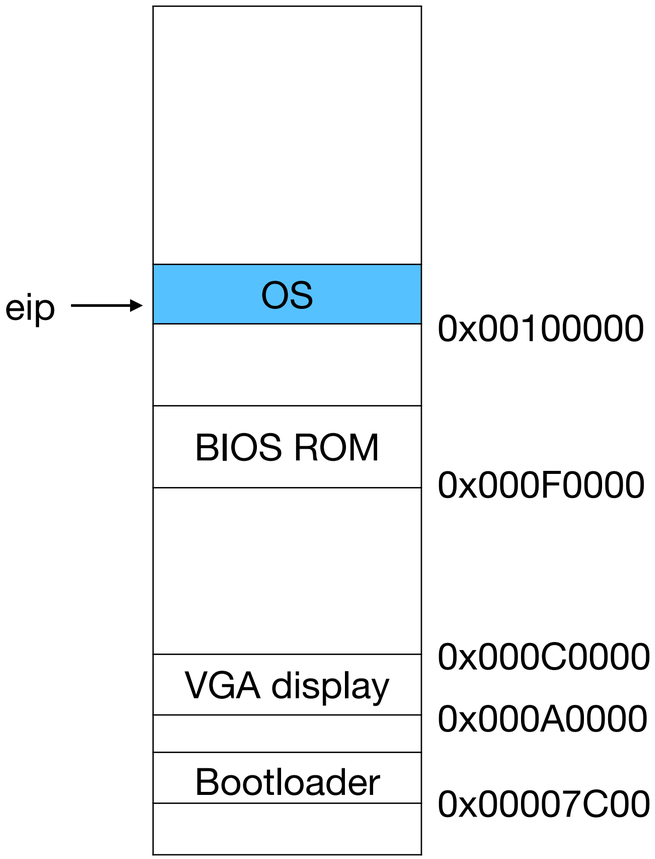
Boot up sequence (3): OS



- gcc prepares OS image in Executable and Linkable Format (ELF)
- Bootloader copies OS image starting from disk sector 1 to 0x00100000 and transfers control to it

Boot up sequence (3): OS





- gcc prepares OS image in Executable and Linkable Format (ELF)
- Bootloader copies OS image starting from disk sector 1 to 0x00100000 and transfers control to it
- We need to tell gcc that OS image will be loaded at 0x00100000

Code walkthrough

p1-booting

Makefile

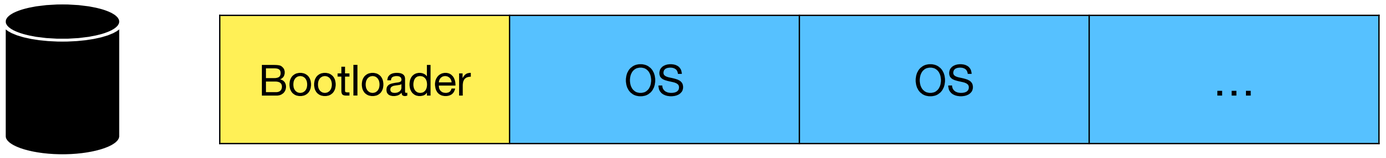
- bootblock.o tells linker that I want to start at 0x7c00
- bootblock pulls out just .text section of bootblock.o
- Kernel tells linker that I want to start at 0x100000 in kernel.ld
- xv6.img copies bootblock at sector 0, kernel starting from sector 1

Makefile

Preparing disk image

```
xv6.img: bootblock kernel
  dd if=/dev/zero of=xv6.img count=10000
  dd if=bootblock of=xv6.img conv=notrunc
  dd if=kernel of=xv6.img seek=1 conv=notrunc
```

xv6.img has bootblock (boot loader) at sector 0, kernel starting at sector 1



Makefile

Preparing bootblock

```
bootblock: bootasm.S bootmain.c
  $(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
  $(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
  $(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
  $(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
  ./sign.pl bootblock
```

- Link bootmain.o and bootasm.o into an ELF executable bootblock.o such that: text section (code) starts at 0x7C00, entry point should be start. See bootblock.asm
- Copy out just the text section from the ELF executable (there is no loader to load the ELF file)
- sign.pl checks that bootblock is less than 510 bytes, adds two special bytes before bootblock to tell the BIOS that it is a valid bootblock

Makefile

Preparing kernel

```
kernel: $(OBJS) entry.o kernel.ld
  $(LD) $(LDFLAGS) -T kernel.ld -o kernel entry.o $(OBJS) -b binary
```

```
# kernel.ld
ENTRY(_start)
SECTIONS
{
  /* Link the kernel at this address: "." means the current address */
  . = 0x00100000;
  .text : AT(0x100000) { .. }
```

- Link entry.o and other object files together to create a kernel image
- Code section (text) starts at 0x00100000 with entry point at _start

Kernel

(p1-booting)$ readelf -l kernel

Elf file type is EXEC (Executable file)

Entry point **0x10000c**

There are **3** program headers, starting at offset **52**

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
LOAD	**0x001000**	0x00100000	**0x00100000**	**0x005A**	0x005A	RE	0x1000
LOAD	0x000000	0x00101000	**0x00101000**	**0x0000**	**0x01000**	RW	0x1000
GNU_STACK	0x000000	0x00000000	0x00000000	0x0000	0x00000	RWE	0x10

Section to Segment mapping:

Segment Sections...

00 .text

01 .bss

02

1. Program headers start from 52

4. Jump to address 0x10000c to start executing

2. Read the text region of size 0x5A from file offset 0x1000 and load it at physical address 0x100000

3. bss region (uninitialised global variables) need not be read since file size is zero. But allocate 0x1000 memory at 0x101000.

Kernel

Byte number 512: second sector of disk

ELF magic header for the OS image

Entry point is 0x0010000C

vim xv6.img
:%!xxd

```
00000200: 7f45 4c46 0101 0100 0000 0000 0000 0000  .ELF............
00000210: 0200 0300 0100 0000 0c00 1000 3400 0000  ............4...
00000220: 3417 0000 0000 0000 3400 2000 0300 2800  4.......4. ...(.
```

vim kernel.asm

```
Disassembly of section .text:
0010000c <_start>:
# Entering xv6 on boot processor, with paging off.
.globl entry
entry:
```

Code walkthrough

p1-booting

Bootloader: bootasm.S

- Starts in 16 bit
- Tell assembler that we are in 16 bit mode
- BIOS may have interrupts enabled, setup its own segment registers
- Clear interrupt flag
- Clear segment registers
- Switch to 32 bit
- Set up stack pointer to grow below .start (stack grows downward)

Code walkthrough (2)

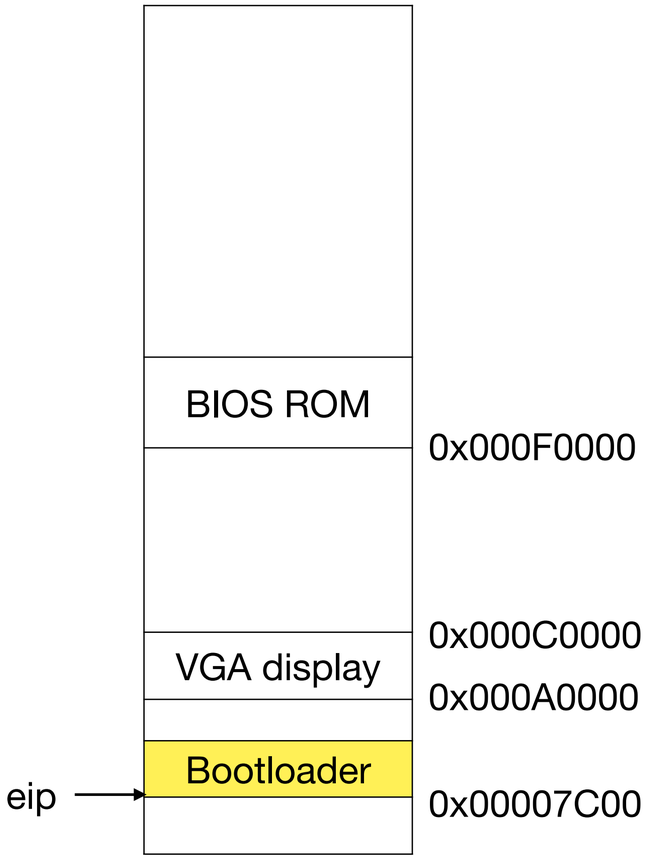
p1-booting

Bootloader: bootasm.S. Switching from 16-bit to 32-bit

- Setup GDT with three entries: null entry, code segment entry, and data/stack segment entry
- LGDT to point GDTR to GDT. The operand of the LGDT instruction is a GDT descriptor gdtdesc, which contains the size of the GDT (first two bytes) and the base address of the GDT (next four bytes).
- Set protected mode bit in CR0 register
- Unlike other selectors, code selector cannot be directly set (similar to how eip cannot be set directly).
- Do ljmp to set code selector
- Tell assembler to start assemble assuming 32 bit mode
- Set data segment, extra segment, and stack segment to data segment. These segments are used by default in data access, string instructions, stack instruction respectively.
- Set FS and GS to null segment. They are never default. Programmer can explicitly use FS, GS. xv6 does not want to use them.

Bootloader bootasm.S

Get to a clean slate from BIOS

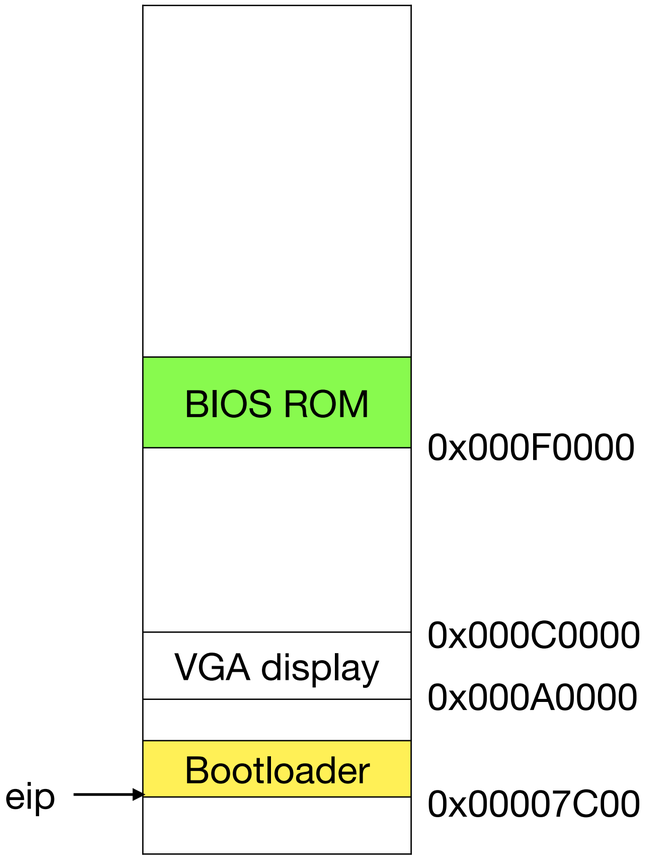



```
.code16        # Assemble for 16-bit mode
.globl start
start:
  cli          # BIOS enabled interrupts; disable

  # BIOS may have set segment registers,
  # zero data segment registers DS, ES, and SS.
  xorw    %ax,%ax              # Set %ax to zero
  movw    %ax,%ds              # -> Data Segment
  movw    %ax,%es              # -> Extra Segment
  movw    %ax,%ss              # -> Stack Segment
```

bootasm.S (2)

Switch to 32-bit,

flat memory model

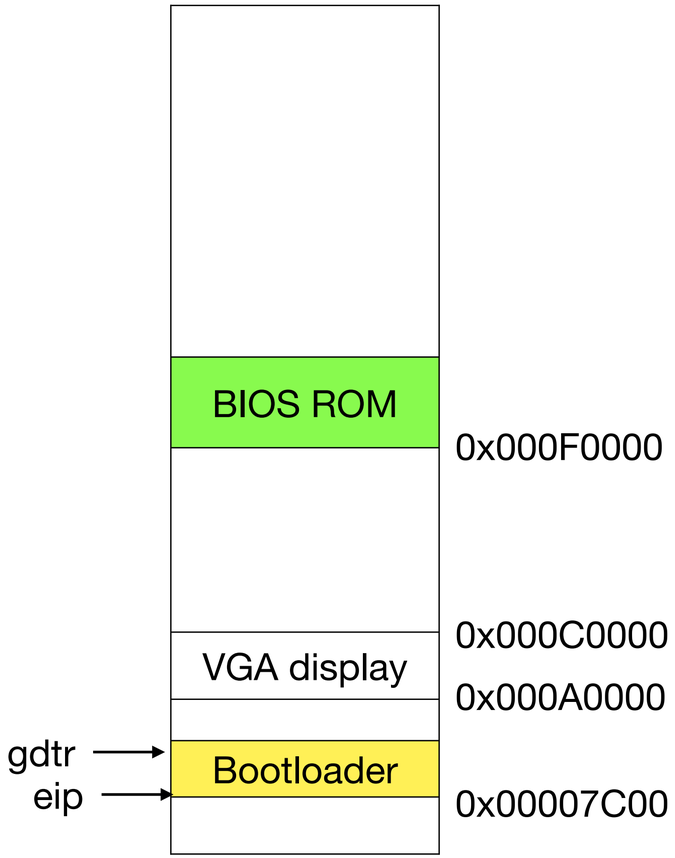



```
  # Switch from real to protected mode.  Use a bootstrap GDT that makes
  # virtual addresses map directly to physical addresses so that the
  # effective memory map doesn't change during the transition.
  lgdt    gdtdesc
  movl    %cr0, %eax
  orl     $CR0_PE, %eax
  movl    %eax, %cr0

  # Complete the transition to 32-bit protected mode by using a long jmp
  # to reload %cs and %eip.  The segment descriptors are set up with no
  # translation, so that the mapping is still the identity mapping.
  ljmp    $(SEG_KCODE<<3), $start32
…

# Bootstrap GDT
.p2align 2                                # force 4 byte alignment
gdt:
  SEG_NULLASM                             # null seg
  SEG_ASM(STA_X|STA_R, 0x0, 0xffffffff)   # code seg
  SEG_ASM(STA_W, 0x0, 0xffffffff)         # data seg

gdtdesc:
  .word   (gdtdesc - gdt - 1)             # sizeof(gdt) - 1
  .long   gdt                             # address gdt
```

bootasm.S (2)

Switch to 32-bit,

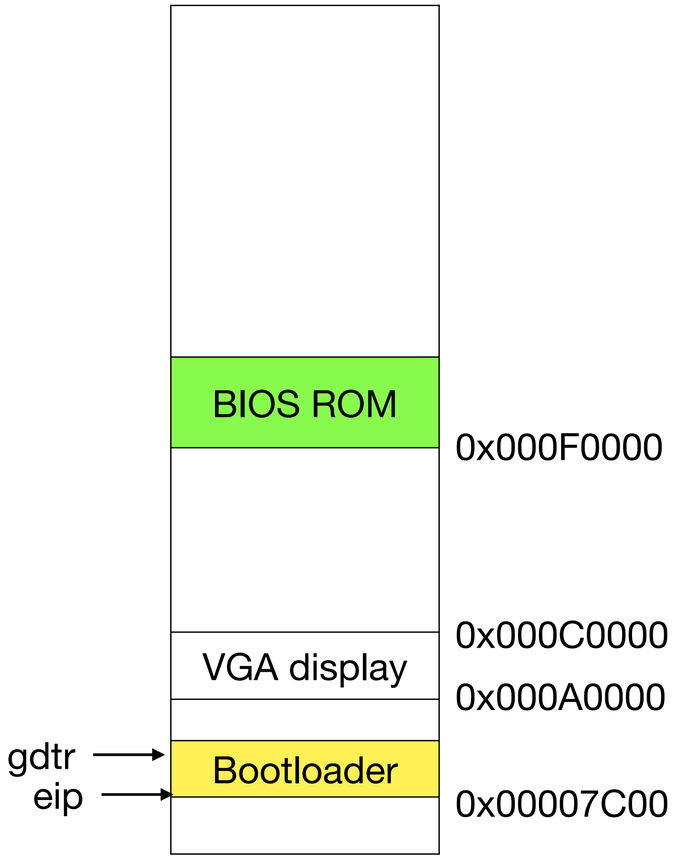
flat memory model




```
  # Switch from real to protected mode.  Use a bootstrap GDT that makes
  # virtual addresses map directly to physical addresses so that the
  # effective memory map doesn't change during the transition.
  lgdt    gdtdesc
  movl    %cr0, %eax
  orl     $CR0_PE, %eax
  movl    %eax, %cr0

  # Complete the transition to 32-bit protected mode by using a long jmp
  # to reload %cs and %eip.  The segment descriptors are set up with no
  # translation, so that the mapping is still the identity mapping.
  ljmp    $(SEG_KCODE<<3), $start32
…

# Bootstrap GDT
.p2align 2                                # force 4 byte alignment
gdt:
  SEG_NULLASM                             # null seg
  SEG_ASM(STA_X|STA_R, 0x0, 0xffffffff)   # code seg
  SEG_ASM(STA_W, 0x0, 0xffffffff)         # data seg

gdtr →  gdtdesc:
  .word   (gdtdesc - gdt - 1)             # sizeof(gdt) - 1
  .long   gdt                             # address gdt
```

bootasm.S (2)

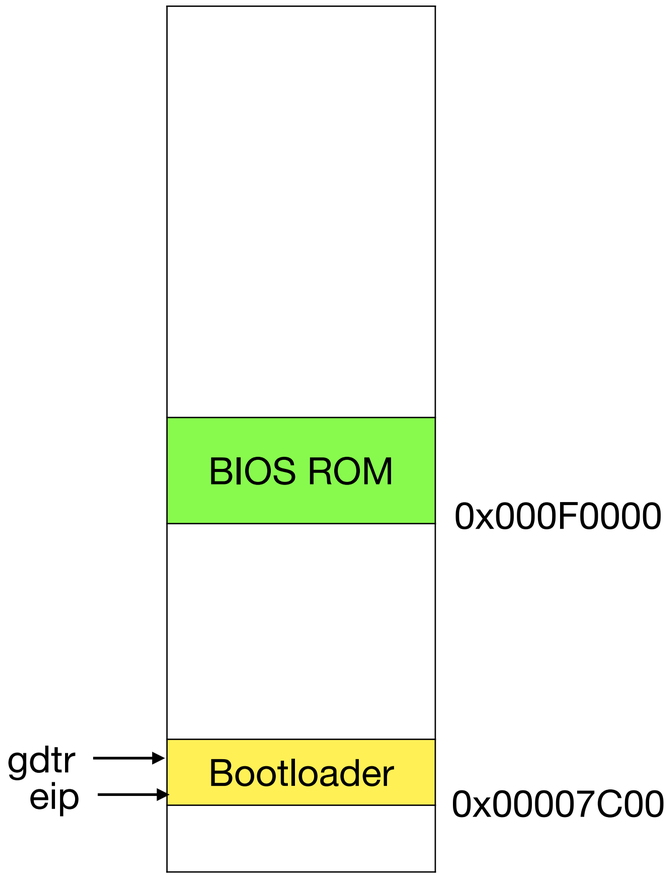
flat memory model




```
  ljmp    $(SEG_KCODE<<3), $start32

.code32  # Tell assembler to generate 32-bit code now.
start32:
  # Set up the protected-mode data segment registers
  movw    $(SEG_KDATA<<3), %ax    # Our data segment selector
  movw    %ax, %ds                # -> DS: Data Segment
  movw    %ax, %es                # -> ES: Extra Segment
  movw    %ax, %ss                # -> SS: Stack Segment
  movw    $0, %ax                 # Zero segments not ready for use
  movw    %ax, %fs                # -> FS
  movw    %ax, %gs                # -> GS
…

# Bootstrap GDT
.p2align 2                                # force 4 byte alignment
gdt:
  SEG_NULLASM                             # null seg
  SEG_ASM(STA_X|STA_R, 0x0, 0xffffffff)   # code seg
  SEG_ASM(STA_W, 0x0, 0xffffffff)         # data seg

gdtr →  gdtdesc:
  .word   (gdtdesc - gdt - 1)             # sizeof(gdt) - 1
  .long   gdt                             # address gdt
```

bootasm.S (3)

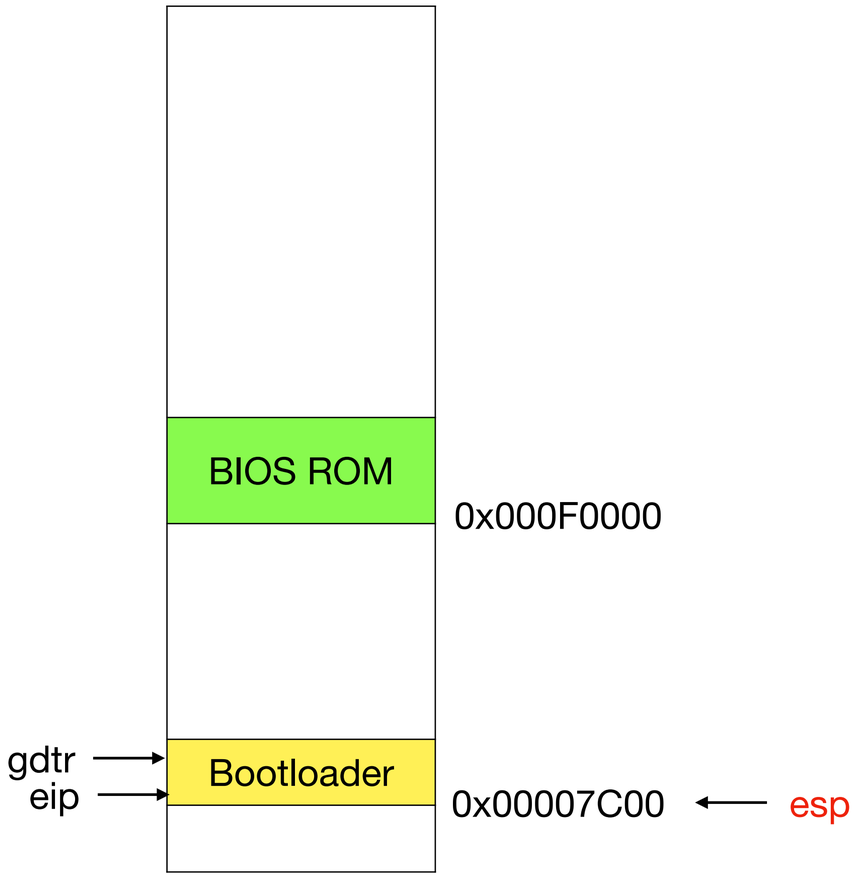
Setup stack pointer, jump to bootmain




```
start32:
  …
  # Set up the stack pointer and call into C.
  movl    $start, %esp
  call    bootmain
```

bootasm.S (3)

Setup stack pointer, jump to bootmain




```
start32:
  …
  # Set up the stack pointer and call into C.
  movl    $start, %esp
  call    bootmain
```

Code walkthrough

p1-booting

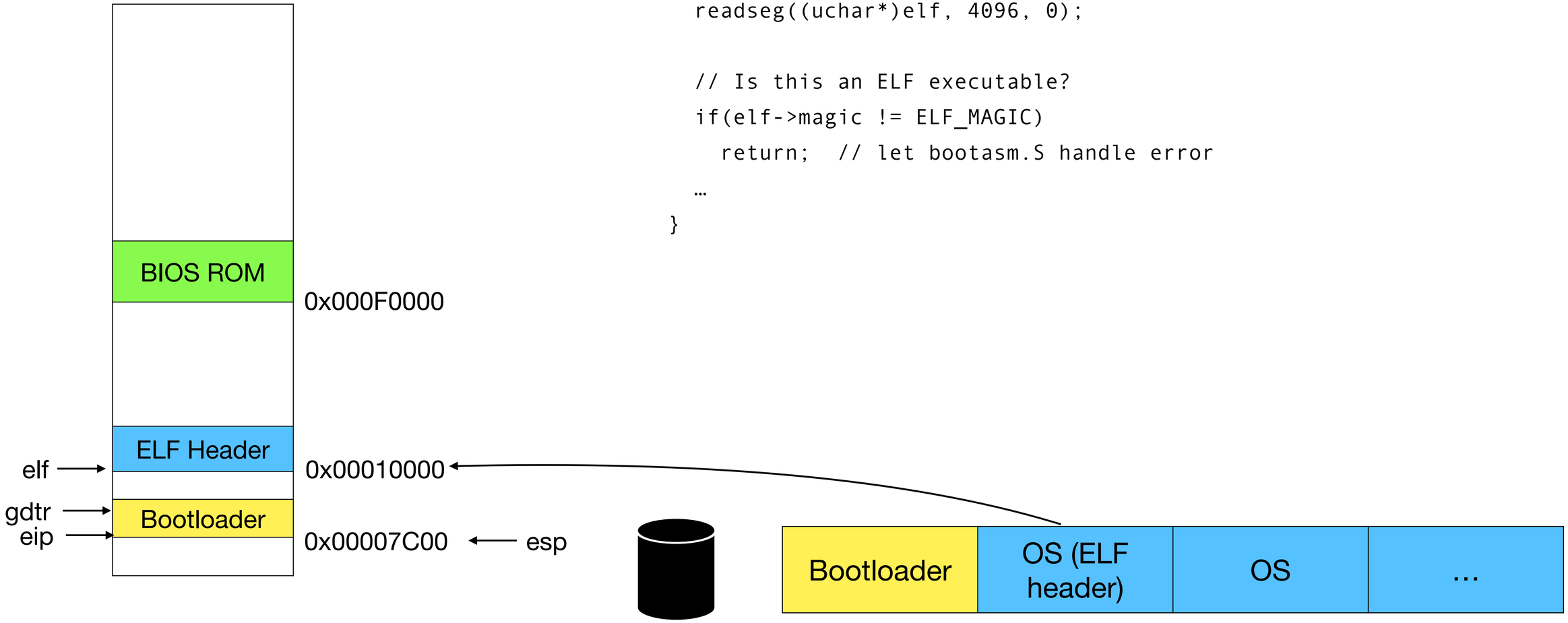
Bootloader: bootmain.c

- bootmain calls readseg to read 4KB (ELF file header)
  - readseg adds 1 since kernel starts at sector 1 and reads sectors 1 by 1.
- bootmain checks if magic header is present. If not, something is wrong: the OS image is corrupted, *etc.* abort
- Read program segment header one by one
  - Read ph->filesz bytes from ph->off on the disk into ph->paddr
  - Zero (memsz - filesz) values from pa + ph->filesz
- Finally jump to the entry point as per ELF

bootmain.c (1)

Read ELF file header

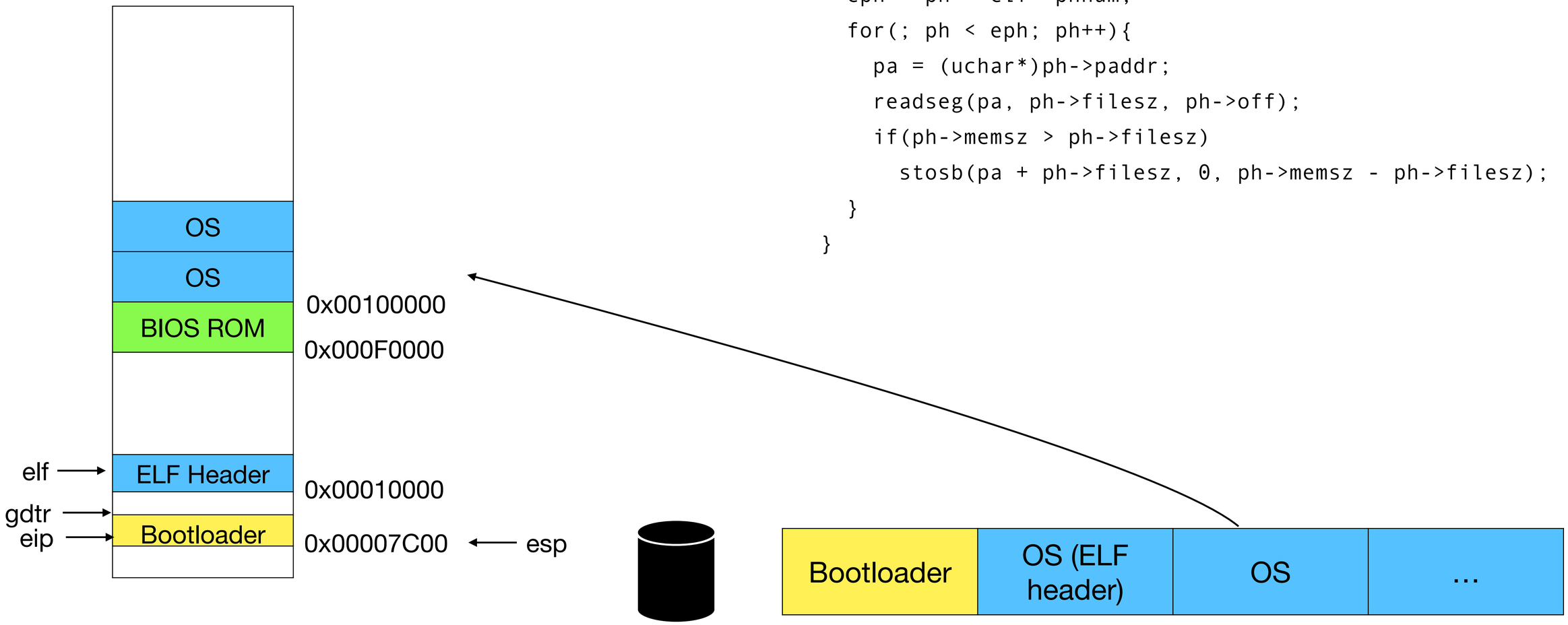
```
void bootmain(void) {
  elf = (struct elfhdr*)0x10000;  // scratch space

  // Read 1st page off disk
  readseg((uchar*)elf, 4096, 0);

  // Is this an ELF executable?
  if(elf->magic != ELF_MAGIC)
    return;  // let bootasm.S handle error
  …
}
```



bootmain.c (2)

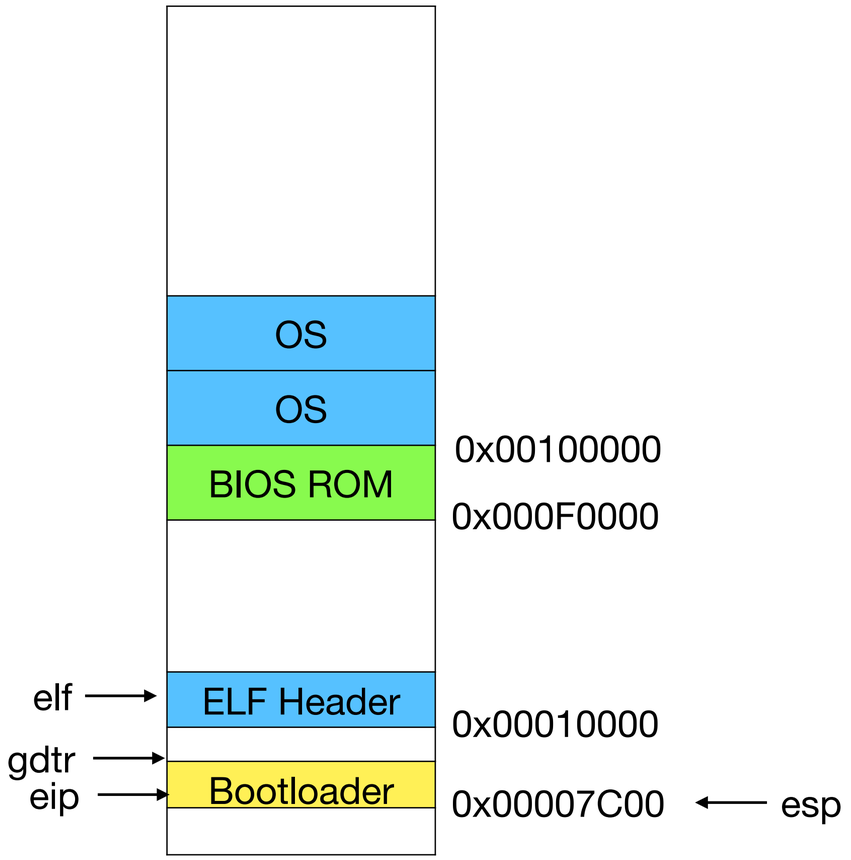
Read segments from ELF file

```
void bootmain(void) {
  ...
  // Load each program segment (ignores ph flags).
  ph = (struct proghdr*)((uchar*)elf + elf->phoff);
  eph = ph + elf->phnum;
  for(; ph < eph; ph++){
    pa = (uchar*)ph->paddr;
    readseg(pa, ph->filesz, ph->off);
    if(ph->memsz > ph->filesz)
      stosb(pa + ph->filesz, 0, ph->memsz - ph->filesz);
  }
}
```



bootmain.c (3)

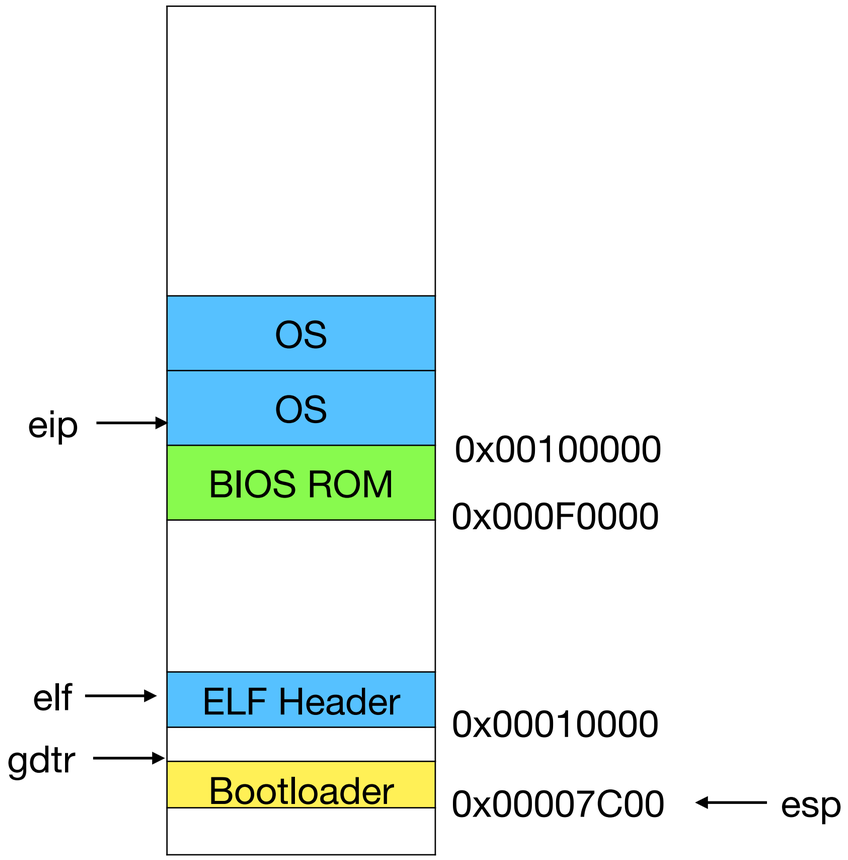
Jump inside OS!




```
void bootmain(void) {
  ...
  // Call the entry point from the ELF header.
  // Does not return!
  entry = (void(*)(void))(elf->entry);
  entry();
}
```

bootmain.c (3)

Jump inside OS!




```
void bootmain(void) {
  ...
  // Call the entry point from the ELF header.
  // Does not return!
  entry = (void(*)(void))(elf->entry);
  entry();
}
```

Code walkthrough

p1-booting

Bootloader: bootmain.c. Reading from disk readseg

- readseg repeatedly calls readsect
- readsect calls waitdisk which waits for the disk to get ready: busy bit is zero and ready bit is one
  - This approach is called *polling*. It wastes CPU cycles since disks can take a long time (~ms). Bootloader has nothing else to do so it is fine here.
- Then it says that I am reading one sector, gives sector offset, and gives command as read
- When the disk is ready, read out 512 bytes using insl from data port to dst

Reading from disk

```
void waitdisk(void) {
  while((inb(0x1F7) & 0xC0) != 0x40);
}
void readsect(void *dst, uint offset) {
  waitdisk();
  outb(0x1F2, 1);   // count = 1
  outb(0x1F3, offset);
  outb(0x1F4, offset >> 8);
  outb(0x1F5, offset >> 16);
  outb(0x1F6, (offset >> 24) | 0xE0);
  // Issue command
  outb(0x1F7, 0x20);
  waitdisk();
  // Read data.
  insl(0x1F0, dst, SECTSIZE/4);
}
```

Register	Direction	Function	Description
0x1F0	R/W	Data register	Read/write data bytes
0x1F2	R/W	Sector count	Number of sectors to read/write
0x1F3-0x1F6	R/W	Sector number	Sector address
0x1F7	R	Status register	Read current status
0x1F7	W	Command register	Send commands to device

Status register (0x1F7)

7	6	5	4	3	2	1	0
BUSY	READY	FAULT	SEEK	DRQ	CORR	IDDEX	ERROR

Head register bits 0x1F6	Function
5, 6, 7	Always set

Command registers	Function
0x20	Read sector
0x30	Write sector

From: https://wiki.osdev.org/ATA_PIO_Mode
https://wiki.osdev.org/ATA_Command_Matrix

Understanding insl

```
asm volatile("cld; rep insl" :
             "=D" (addr), "=c" (cnt) :
             "d" (port), "0" (addr), "1" (cnt) :
             "memory", "cc");
```

```
asm volatile("cld; rep insl" :
  7ce8: 8b 7d 08        mov    0x8(%ebp),%edi
  7ceb: b9 80 00 00 00  mov    $0x80,%ecx
  7cf0: ba f0 01 00 00  mov    $0x1f0,%edx
  7cf5: fc              cld
  7cf6: f3 6d           rep insl (%dx),%es:(%edi)
```

```
cld;
i=0;
while(i++ < ecx) {
     insl %dx, %edi
}
```

- cld: clear direction flag (a bit in EFLAGS): read in forward direction. (std for reads in backward direction)
- “=D” (adds): move addr into edi
- “=c” (cnt): move cnt into ecx
- “d” (port): move port into edx
- memory: instruction will clobber memory
- cc: instruction will clobber condition code (EFLAGS) register
- asm volatile: don’t optimise it away

Code walkthrough

p1-booting

OS: entry.S, main.c

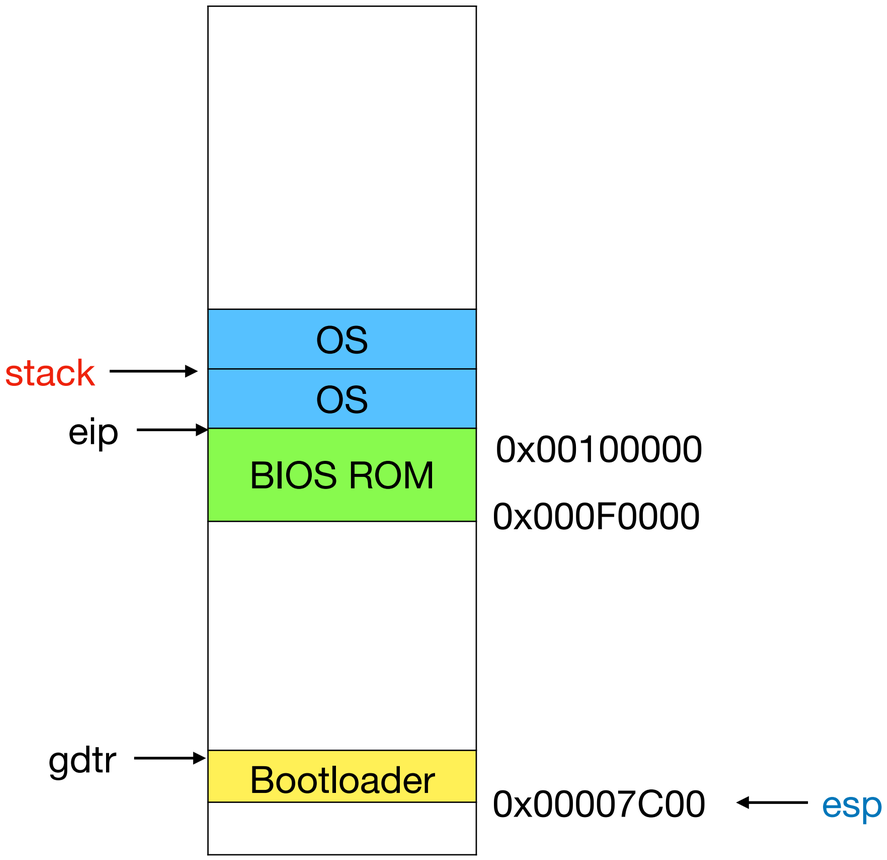
- entry.S allocated 4KB space in the bss section with “.comm stack, KSTACKSIZE”

- Then it sets the stack pointer to the top of the allocation (since stack grows downwards) and calls main.c

- main.c sends shutdown signal to QEMU (https://wiki.osdev.org/Shutdown)

entry.S

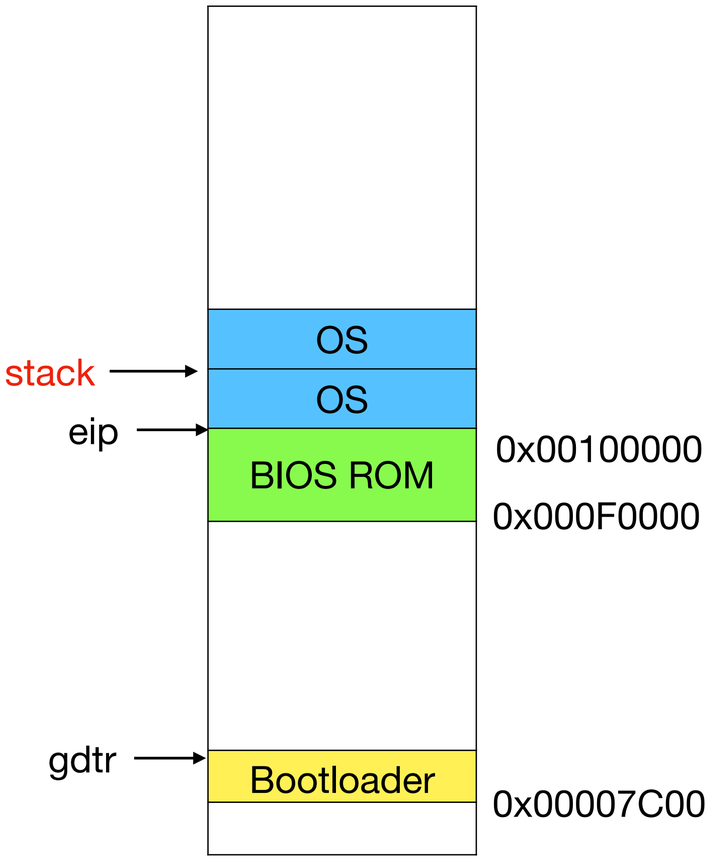
Setup the stack, jump into main




```
.globl _start
_start = entry

# Entering xv6 on boot processor, with paging off.
.globl entry
entry:
  # Set up the stack pointer.
  movl $(stack + KSTACKSIZE), %esp
  call main

.comm stack, KSTACKSIZE
```

entry.S

Setup the stack, jump into main

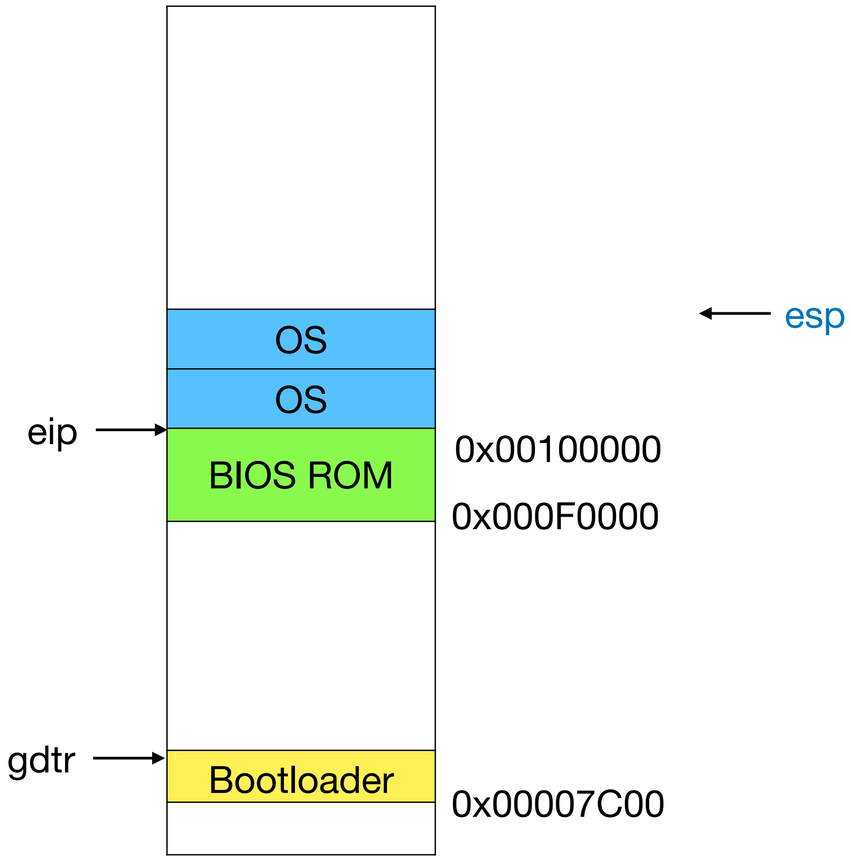


esp

```
.globl _start
_start = entry

# Entering xv6 on boot processor, with paging off.
.globl entry
entry:
  # Set up the stack pointer.
  movl $(stack + KSTACKSIZE), %esp
  call main

.comm stack, KSTACKSIZE
```

main.c

Send shutdown signal




```
int halt(void) {
  outw(0x604, 0x2000);
  // For older versions of QEMU,
  outw(0xB004, 0x2000);
  return 0;
}

// Bootstrap processor starts running C code here.
int main(void) {
  halt();
}
```

Modern systems use two-step booting procedure

- Our Bootloader fits in 512 bytes and assumes there is one OS contiguously stored from sector 1
  - Bootloader boots into a temporary OS
  - Temporary OS can download OS from network, boot from USB, etc. i.e, it is a full-fledged OS with support for reading file systems, TCP/IP to talk to network, etc.